

# Matrix Commitment-Based Ownership Verification for Distributed Machine Learning

Tianxiu Xie<sup>ID</sup>, Keke Gai<sup>ID</sup>, *Senior Member, IEEE*, Jing Yu<sup>ID</sup>, Liehuang Zhu<sup>ID</sup>, *Senior Member, IEEE*,  
and Qi Wu<sup>ID</sup>, *Senior Member, IEEE*

**Abstract**—Distributed Machine Learning (DML) enables parallel training but creates conflicts regarding model ownership determination, as verifying the specific contributions of individual clients during weight updates remains a challenge. To resolve ownership disputes and prevent malicious clients from forging data, verifying the integrity and effectiveness of each client’s unique training history can fairly establish their partial ownership of the final DML model. In this paper, for the first time we propose a novel *Matrix Commitment-based DML Ownership Verification* (MAMMON) scheme that adopts a concise proof to ensure computational integrity and correctness within a limited computing cost. In contrast to computationally intensive SNARK-based approaches, MAMMON constructs a multi-linear tree structure to reduce proof update costs and avoids complex arithmetic circuit computations by standardizing the training process. Additionally, we watermark the weight proofs using client identity private keys to safeguard the commitments against tampering or unauthorized acquisition. Extensive experiments demonstrate MAMMON’s superior performance in preserving computational integrity of DML training, covering both theoretical analysis and comparisons with other existing baselines.

**Index Terms**—Matrix commitment, ownership verification, distributed machine learning, intellectual property protection.

## I. INTRODUCTION

WHILE Large Language Models (LLMs) have revolutionized language tasks, their prohibitive training costs [1],

Received 11 December 2024; revised 13 February 2026; accepted 21 April 2026. Date of publication 27 April 2026; date of current version 6 August 2026. This work was supported in part by the National Natural Science Foundation of China under Grant U24B20146 and Grant 62372044, in part by Beijing Municipal Science and Technology Commission Project under Grant Z241100009124008, and in part by the Beijing Nova Program under Grant 20250484921. Recommended for acceptance by J. Cai. (Corresponding authors: Keke Gai; Jing Yu.)

Tianxiu Xie and Liehuang Zhu are with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: 3120215672@bit.edu.cn; liehuangz@bit.edu.cn).

Keke Gai is with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China, also with the School of AI, Beijing Institute of Technology, Beijing 100081, China, and also with the Zhongguancun Academy, Haidian, Beijing 100190, China (e-mail: gaikeke@bit.edu.cn).

Jing Yu is with the Key Laboratory of Ethnic Language Intelligent Analysis and Security Governance of MOE, Minzu University of China, Beijing 100081, China, and also with the School of Information Engineering, Minzu University of China, Beijing 100081, China (e-mail: jing.yu@muc.edu.cn).

Qi Wu is with the Australian Institute for Machine Learning (AIML), School of Computer Science, The University of Adelaide, Adelaide, SA 5005, Australia (e-mail: qi.wu01@adelaide.edu.au).

The code is available at <https://github.com/tx123456/VCDmIOP-test>.  
Digital Object Identifier 10.1109/TPAMI.2026.3687640

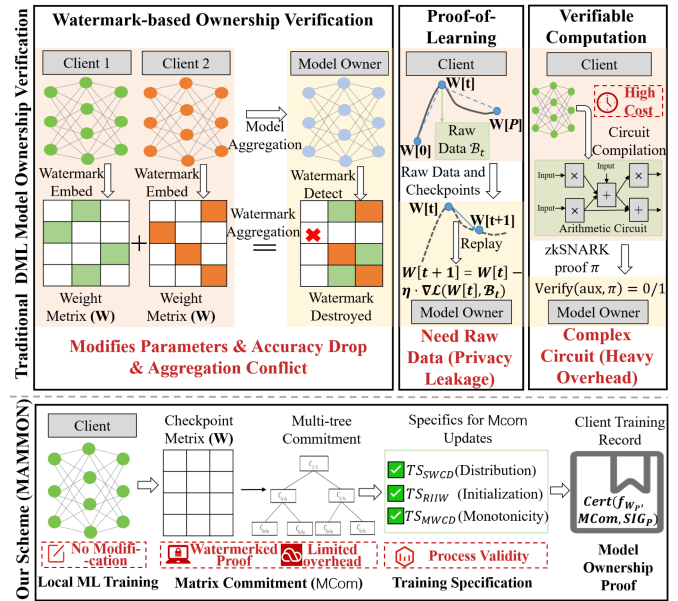


Fig. 1. An illustration of the motivation of our Matrix Commitment-based DML Ownership Verification scheme.

[2] necessitate Distributed Machine Learning (DML) as a solution for efficient scaling across dynamic clients [3]. However, the dynamic nature of DML introduces critical conflicts regarding model ownership verification [4]. Specifically, once the final model weights are released, adversaries may misappropriate the ownership of correct clients because proving contributions to the final model remains a challenge [5], [6], [7]; simultaneously, model owners must verify the integrity and correctness of each client’s computation to prevent malicious actors from stealing or forging model weights during training and aggregation. Consequently, ensuring computational integrity and correctness is crucial for achieving robust ownership verification of every honest client in DML.

Most existing schemes for ownership verification in DML are encountering challenges from either adversaries or efficiency restrictions. Fig. 1 shows the limitations of traditional DML ownership verification schemes, illustrating the motivation for our MAMMON. Three fundamental types are synthesized below. The first is model watermarking-based methods that embeds trigger samples or feature patterns into the global/local model and queries the suspicious model to check ownership [8], [9], [10], [11], [12], [13]. However, typical DML watermarking schemes either only support embedding a single watermark into

the global model [9], [10], or they render client-specific private watermarks susceptible to exposure and mutual interference during aggregation [11], [12], [13], which makes it difficult to prove that a particular client actually contributed to the final model. Second, computation-based model ownership verification schemes ensure the computational integrity of clients by replaying the training process with intermediate models and the associated training data points [14], [15], [16], [17], [18]. Unfortunately, this type of scheme may force clients to disclose raw data batches [15], [16], [17], and the replay process may be attacked by forged or adversarial examples [19], [20]. To avoid adversarial attacks, the third typical ownership verification scheme is cryptographic approaches based on verifiable computation and commitments [21], [22], [23], [24], [25]. Although verifiable computation can support zero-knowledge ownership verification for all ML models, the required arithmetic lifting of DML operations and per-update proof refresh may result in substantial computational overhead for clients [16], [26].

Therefore, a core issue in DML is to find out an effective proving scheme with limited overhead and concise proof during DML training. Specifically, effective proofs need to satisfy: (i) **Training integrity**, verifying clients' genuine resource usage; (ii) **Execution correctness**, ensuring effective computation of training tasks. To address issues above, in this paper we propose a novel *Matrix Commitment-based DML Ownership Verification* (MAMMON) scheme. In each global DML epoch, clients locally optimize parameters using stochastic gradient-based algorithms (e.g., SGD). To mitigate expensive arithmetic overhead, our scheme generates commitment proofs solely for model weight checkpoint matrices rather than the entire model. Inspired by *PoT* [27] and *RAI*<sup>2</sup> [28], we record weight updates in a matrix ( $\mathbf{W}$ ), representing the client's unique training history. Clients generate a matrix commitment (MCom) and certificate for the weight parameter matrix ( $\mathbf{W}$ ), which aggregates updates from intermediate checkpoints. Crucially, our scheme supports sub-linear time updates for MCom proofs as weights change, avoiding recomputation from scratch. We construct the concept of "*Client Training Record*" (CTR) as the minimal unit of model ownership operations in MAMMON. An effective CTR, which includes the commitment certificate, represents the necessary optimization computations performed by the client in every global epoch. By leveraging the unique training history in  $\mathbf{W}$ , the CTR guarantees the auditability of the client's ownership of the final DML model. Specifically, the matrix commitment scheme leverages *Binding* and *Hiding* properties to establish training integrity for ownership verification. This ensures that a client's committed local model verifiably contributes to the final DML model, utilizing MCom as proof of partial ownership.

However, commitment alone cannot prevent adversaries from fabricating weights without actual training. To address this, we introduce "Training Specification" (TS) to model the update logic of the weight matrix ( $\mathbf{W}$ ) and govern MCom operations. TS ensures execution correctness by enforcing verifiable constraints on model checkpoints, certifying they derive from actual training on private data rather than synthesis. Thus, the commitment binds local model states, while TS authenticates the legitimacy of the training history. Together, MCom and TS

guarantee computational integrity and correctness while preserving data zero-knowledge. To prevent forgery, the client's identity key serves as a watermark binding the commitment. A trusted model owner verifies these watermarks and certificates, which are stored on a distributed ledger (e.g., blockchain). The ledger's timestamps ensure spatiotemporal properties for copyright protection. Notably, MAMMON targets verifiable training history and cryptographic provenance rather than global model watermarking. These commitments enable the owner to prove the DML model incorporates legitimate client contributions, thereby supporting IP protection.

Our main contributions are summarized as follows:

- 1) We propose MAMMON, the first effective DML multi-client ownership verification framework ensuring computational integrity without compromising model structure or data privacy. To ensure training correctness with limited update and verification costs, MAMMON preserves original architectures and optimization algorithms, incorporating commitments strictly for computational integrity. Due to the novel multi-linear tree structure, we implement an aggregable and maintainable matrix commitment scheme to facilitate multi-client DML ownership auditing.
- 2) We propose the first non-stealable and homomorphic DML ownership proof via the multi-linear tree structure. By embedding identity watermarks into commitments, MAMMON ensures non-stealability against proof theft (e.g., signature replacement) without hindering operations. Crucially, our homomorphism-based commitment scheme achieves privacy-preserving aggregation without compromising verification efficiency.
- 3) Our scheme models the process of weight checkpoint changes by attaching specific TSs to CTRs, enabling auditable local ML training. Unlike verifiable computation, TS satisfiability eliminates the need for model and dataset proofs, avoiding expensive generation costs.
- 4) We evaluate MAMMON's aggregation, update, and verification costs on classic ML models to demonstrate its effectiveness for DML copyright verification. Furthermore, our matrix commitment scheme significantly outperforms SOTA methods (Poseidon, Pedersen) in parameter, commitment, and proof generation times.

The organization of this paper follows the order below. The preliminaries of commitment is given in Section II. Sections III and IV present the design of the proposed MAMMON and explanations of multi-tree-based matrix commitment, respectively. Sections V and VI provide security analysis of the proposed model and experiment evaluations with findings, respectively. Related work is given in Section VII. Finally, we discuss the limitation and future work in Section VIII, and conclude this paper in Section IX.

## II. PRELIMINARIES

*Notation:* Matrices, vectors, and scalars are denoted by bold uppercase, bold lowercase, and lowercase letters, respectively. In this paper,  $\mathbf{W}$  represents the client's weight matrix, where the  $i$ -th row  $\mathbf{w}_i$  corresponds to the weight vector of the  $i$ -th layer,

and the entry  $w_{i,j}$  denotes the  $j$ -th parameter of that layer.  $\mathbf{W}[t]$  indicates the weight matrix at the  $t$ -th local epoch. Furthermore,  $\mathbb{F}$  and  $\mathbb{Z}$  denote a finite field and the set of integers, respectively, while  $[N]$  represents the set  $\{1, \dots, N\}$ .

*Multi-linear Extension:* Any function  $f: 0, 1^h \rightarrow \mathbb{F}$  admits a unique multi-linear extension  $\tilde{f}: \mathbb{F}^h \rightarrow \mathbb{F}$  such that  $\tilde{f}(\mathbf{x}) = f(\mathbf{x})$  for all  $\mathbf{x} \in \{0, 1\}^h$ .

*Definition II.1:*  $\forall f: \{0, 1\}^h \rightarrow \mathbb{F}$ ,  $\exists$  a unique multi-linear extension polynomial  $\tilde{f}$  is defined as follows where  $\alpha_i$  and  $\beta_i$  denote the  $i$ -th bits of the vectors  $\alpha$  and  $\beta$ , respectively:

$$\tilde{f}(\alpha) = \sum_{\beta \in \{0,1\}^h} \prod_{i=1}^h ((1 - \alpha_i)(1 - \beta_i) + \alpha_i \beta_i) \cdot f(\beta)$$

*Commitment Scheme:* A commitment scheme  $\text{CS} = (\text{CS.Gen}, \text{CS.Commit}, \text{CS.Open})$  is a protocol between a prover  $\mathcal{P}$  and a verifier  $\mathcal{V}$  consisting of three PPT algorithms:

- $\text{ppc} \leftarrow \text{CS.Gen}(1^\lambda)$ : Generates public parameters  $\text{ppc}$  based on the security parameter  $\lambda$ .
- $\text{com} \leftarrow \text{CS.Commit}_{\text{ppc}}(Mg; cr)$ :  $\mathcal{P}$  computes the commitment  $\text{com}$  using message  $Mg$  and randomness  $cr$ .  $\text{com}$  is sent to  $\mathcal{V}$  while keeping  $Mg$  hidden.
- $\{0, 1\} \leftarrow \text{CS.Open}_{\text{ppc}}(\text{com}, Mg; cr)$ :  $\mathcal{P}$  reveals  $Mg$  and  $cr$ .  $\mathcal{V}$  verifies their consistency with  $\text{com}$ , outputting 1 (ACCEPT) or 0 (REJECT).

The commitment scheme possesses both *Binding* and *Hiding* security properties as follows:

- *Binding*: Finding a collision is computationally infeasible for any PPT adversary. The probability of finding distinct openings  $(Mg, cr)$  and  $(Mg', cr')$  (where  $Mg \neq Mg'$ ) that verify to the same commitment  $\text{com}$  is negligible.
- *Hiding*: The commitment reveals nothing about the message. For any equal-length messages  $Mg, Mg'$ , the distributions of commitments are computationally indistinguishable as follows, denoted by  $cr \xleftarrow{\$} R_{\text{ppc}}$  and  $\approx_c$ :

$$\{\text{CS.Commit}_{\text{ppc}}(Mg; cr)\} \approx_c \{\text{CS.Commit}_{\text{ppc}}(Mg'; cr)\}.$$

*Trapdoor Commitment:* The trapdoor commitment scheme TCS extends the standard CS over  $\mathbb{F}$  with two auxiliary algorithms, denoted as  $\text{TCS} = (\text{TCS.Gen}, \text{TCS.Commit}, \text{TCS.Open}, \text{TCS.TrapGen}, \text{TCS.TrapOpen})$ :

- $(\text{pp}_{\text{TCS}}, \text{tm}_{\text{TCS}}) \leftarrow \text{TCS.TrapGen}(1^\lambda)$ : Outputs public parameters  $\text{pp}_{\text{TCS}}$  and a trapdoor  $\text{tm}_{\text{TCS}}$ .
- $cr' \leftarrow \text{TCS.TrapOpen}_{\text{pp}_{\text{TCS}}}(\text{tm}_{\text{TCS}}, \text{com}, Mg; cr)$ : Using the trapdoor  $\text{tm}_{\text{TCS}}$ , this algorithm computes a randomness  $cr'$  such that  $\text{com}$  is a valid commitment to  $Mg$  (i.e.,  $\text{TCS.Commit}_{\text{pp}_{\text{TCS}}}(Mg; cr') = \text{com}$ ).

*Principal Component Analysis (PCA):* PCA is a linear dimensionality reduction technique that projects features onto an orthogonal basis ordered by variance. Considering parameter independence, uniform feature contributions and dominant correlations absence disperse variance across multiple components, preventing a disproportionate concentration of variance in the first principal component.

### III. DEFINITIONS AND METHODOLOGY

#### A. Overview of MAMMON

MAMMON prioritizes training integrity over inference. In each global epoch, clients generate CTRs containing matrix commitments (MCom) of the weight matrix  $\mathbf{W}$ . These CTRs ensure execution correctness via three training specifications and allow opening specific  $\mathbf{W}$  positions. Consequently, our scheme addresses: (i) verifying the correctness of per-epoch local training results ( $f_{W_P}^a$ ), and (ii) enabling clients to claim partial ownership of the final DML model ( $f_{W_P}$ ).

We assume that the client involved in distributed learning is prover  $\mathcal{P}$ , the model owner is verifier  $\mathcal{V}$ , and the malicious client is adversary  $\mathcal{A}$ . During local ML training,  $\mathcal{P}$  records the intermediate model parameters at intervals and synchronously updates the weight parameters on the corresponding matrix ( $\mathbf{W}$ ). Let  $\mathbf{W} = [\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_L]^\top$  denote the weight matrix of an  $L$ -layer model, where for the  $i$ -th layer ( $i \in [L]$ ),  $\mathbf{w}_i = [w_{i,1}, w_{i,2}, \dots, w_{i,N}]$  denotes its weight vector. In one global epoch, a client  $\mathcal{P}$  first initials the local ML model and records the initial weight values on the matrix ( $\mathbf{W}$ ). Subsequently,  $\mathcal{P}$  generates a commitment (MCom) to the model weight matrix  $\mathbf{W}$ . During one local epoch of  $\mathcal{P}$ , each weight checkpoint is validated against three TSs. Upon successful validation of all TSs,  $\mathcal{P}$  proceeds to update the commitment (MCom) and generates a corresponding verifiable proof. This ensures that each accepted local ML model is cryptographically bound to  $\mathcal{P}$ 's identity remains immutable under the commitment, while the matrix ( $\mathbf{W}$ ) remains hidden until explicitly opened. Consequently,  $\mathcal{V}$  is able to efficiently ascertain both the inclusion and the update of the client's ML model. One CTR records the commitment certificate  $\text{Cert}(f_{W_P}, \text{MCom}, \text{SIG}_{\mathcal{P}})$  of  $\mathcal{P}$ , where  $f_{W_P}$  represents the final DML model trained by  $\mathcal{P}$ , and  $\text{SIG}_{\mathcal{P}}$  represents the signature of  $\mathcal{P}$ . Specifically, as model checkpoints induce iterative changes to  $\mathbf{W}$ , MAMMON supports sub-linear-time updates to the matrix commitment (MCom). In local ML training, without any additional knowledge of  $\mathcal{P}$ 's dataset, CTR prevents  $\mathcal{A}$  from forging weight parameters through ensuring three TSs' satisfiability.

In MAMMON, each per-local-epoch weight checkpoint triggers an update to the matrix commitment MCom embedded in  $\text{Cert}(\cdot)$  and to the corresponding CTR binding commitment CP (Def. III.1). We record these commitment updates on a publicly verifiable, append-only distributed ledger (e.g., blockchain), so that verifiers can unambiguously determine the latest committed state and detect any attempts to present stale states (replay/rollback) or to omit required updates (selective submission) by auditing the ledger history.

#### B. Security for MAMMON

In MAMMON, the verifier  $\mathcal{V}$  is honest and faithfully executes the DML training. A malicious client  $\mathcal{A}$  attempts to mislead  $\mathcal{V}$  by falsely claiming that it has correctly and completely executed the training of  $f_{W_P}$ . Its goal is to seize ownership of the final DML model by forging a certificate  $\text{Cert}(f_{W_P}, \text{MCom}, \text{SIG}_{\mathcal{A}})$  without performing the correct local training required to generate  $f_{W_P}^a$ .



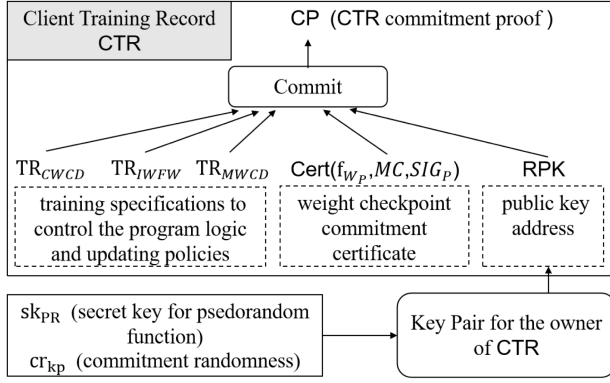
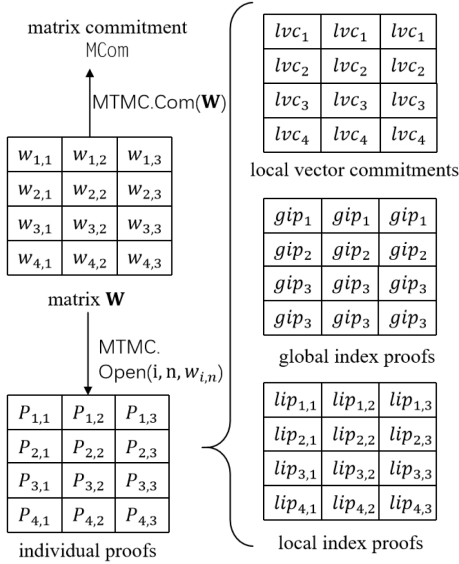


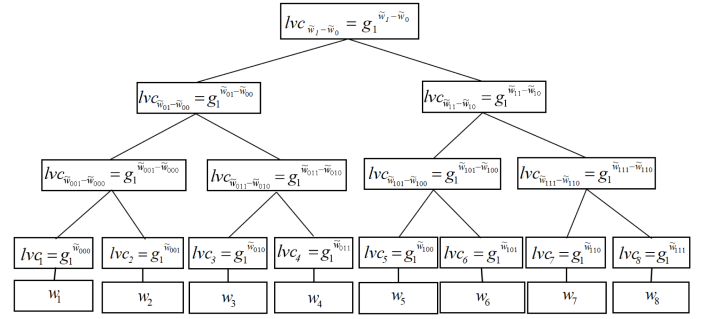
Fig. 2. The construction of Client Training Record.

Fig. 3. Matrix Commitment for  $(L, N) = (4, 3)$ .

**System Trust Assumptions:** Beyond the cryptographic binding and hiding guarantees of MCom and CP, MAMMON assumes an append-only log with a total order over finalized entries for publishing and persisting commitment updates. The log is instantiated by a distributed ledger and provides the latest finalized state for each CTR. The distributed ledger provides robust defenses against system-level adversarial behaviors, including replay, rollback, and selective submission. By validating freshness against the latest finalized ledger state, the system fundamentally rejects outdated inputs resulting from replay or rollback attacks. Furthermore, any attempt at selective submission violates the integrity of the update sequence, as evidenced by missing entries in the ledger.

**Threat Model:** We assume that  $\mathcal{A}$  has white-box access to the  $f_{W_P}$  structure and weights, and consider  $\mathcal{A}$ 's ability to effectively deceive  $\mathcal{V}$  from the following five aspects:

- M1: Since  $\mathcal{A}$  has knowledge of the final weight parameters  $\mathbf{W}[P]$  of the local ML model, they can intentionally set the initial weight  $\mathbf{W}[0]$  parameters close to  $\mathbf{W}[P]$ , instead of random initialization. By placing  $\mathbf{W}[0]$  in a favorable position,  $\mathcal{A}$  can bias the ML training process

Fig. 4. An example of the multi-linear tree with the size of 8. Each node represents a commitment. All nodes in the path from  $w_{i,n}$  to root represents the proof for  $w_{i,n}$ .TABLE I  
COMPUTATIONAL COMPLEXITY AMONG DIFFERENT DML OWNERSHIP VERIFICATION SCHEMES

| Scheme          | Proof Generation                                                         | Proof Updates                                                            | Proof Verification  |
|-----------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------|---------------------|
| Merkle SNARK    | $O(M)$                                                                   | $O(M)$                                                                   | $O(M)$              |
| Hyperproof [30] | $(M) \cdot O(M)$                                                         | $O(\log(M))$                                                             | $O(\log(M))$        |
| zkPoT [26]      | $O\left(\begin{matrix} S \cdot M \\ \log(S \cdot M) \end{matrix}\right)$ | $O\left(\begin{matrix} S \cdot M \\ \log(S \cdot M) \end{matrix}\right)$ | $O(S \cdot M)$      |
| PoLO [33]       | $O(M)$                                                                   | $O(M)$                                                                   | $O(C_{fwd}) + O(M)$ |
| MAMMON          | $O(M)$                                                                   | $O(L + \log N)$                                                          | $O(L \cdot \log N)$ |

TABLE II  
MODEL ARCHITECTURES AND PARAMETER SCALES ( $N$ )

| Model                 | Architecture (brief)                 | $N$              |
|-----------------------|--------------------------------------|------------------|
| LeNet                 | 2 Conv + 3 FC                        | $\approx 2^{16}$ |
| ResNet18              | 8 ResBlk + stem + FC                 | $\approx 2^{25}$ |
| AlexNet               | 5 Conv + 3 FC                        | $\approx 2^{26}$ |
| VGG16                 | 13 Conv + 3 FC                       | $\approx 2^{27}$ |
| CLIP                  | ViT-B/32 + Transformer text          | $\approx 2^{28}$ |
| BERT-Large            | 24 Transformer encoder (hidden 1024) | $\approx 2^{29}$ |
| Stable Diffusion v1.4 | Latent UNet + VAE + Text encoder     | $\approx 2^{30}$ |
| Whisper-Large         | Encoder-decoder Transformer          | $\approx 2^{31}$ |

towards their desired final result  $f_{W_P}$  and fabricate a training history to deceive  $\mathcal{V}$ .

- M2: Once current weight checkpoint is released,  $\mathcal{A}$  synthesize a batch of training data points or reorder the local data to manipulate the learning direction and force gradient updates toward an adversary-chosen target. In our threat model, the purpose of such synthesis or reordering is not to corrupt the global DML model or impede convergence, but to economize computation and forge training histories that converge (starting from the same initialization) to a targeted  $f_{W_P}$  without performing genuine training. This manipulation yields non-unique and plausible training histories that can achieve end performance comparable to  $f_{W_P}$  while having a different provenance, undermining model ownership verification.
- M3:  $\mathcal{A}$  fabricates intermediate checkpoints for illicitly acquired models by using linear interpolation between the parameters of a randomly initialized model and those pilfered from the target model at varying proportions.
- M4:  $\mathcal{A}$  aims to remove the cryptographic signature on the  $\text{Cert}(f_{W_P}, \text{MCom}, \text{SIG}_P)$  and replace it with its own

TABLE III

SINGLE-THREADED EXPERIMENTAL EVALUATION OF MAMMON FOR MTMC SCHEME. *OP.* AND *VERI.* DENOTES THE *OPERATION* AND *VERIFICATION*, RESPECTIVELY. *MTMC.ComUpdate OP.* AND *MTMC.ProofAllUpdate OP.* IS THE TOTAL TIME COSTS FOR  $\eta = 1024$  CHANGES TO THE WEIGHT VECTOR. *MTMC.AggRow OP.* DENOTES THE TIME COST TO AGGREGATE THE MAXIMUM NUMBER OF PROOF AT  $\eta = 2^h$ .

| MAMMON Performance for MTMC |                                  | $L=N=4$<br>$h=2$ | $L=N=8$<br>$h=3$ | $L=N=16$<br>$h=4$ | $L=N=32$<br>$h=5$ | $L=N=64$<br>$h=6$ | $L=N=128$<br>$h=7$ | $L=N=256$<br>$h=8$ | $L=N=512$<br>$h=9$ |
|-----------------------------|----------------------------------|------------------|------------------|-------------------|-------------------|-------------------|--------------------|--------------------|--------------------|
| Proof Generation            | MTMC.Gen Op.                     | 0.006(s)         | 0.014(s)         | 0.044(s)          | 0.153(s)          | 0.621(s)          | 2.522(s)           | 10.141(s)          | 38.496(s)          |
|                             | MTMC.Com Op.                     | 0.001(s)         | 0.002(s)         | 0.009(s)          | 0.035(s)          | 0.139(s)          | 0.670(s)           | 2.399(s)           | 9.033(s)           |
|                             | MTMC.Open Op.                    | 0.001(s)         | 0.003(s)         | 0.009(s)          | 0.036(s)          | 0.141(s)          | 0.615(s)           | 2.384(s)           | 8.935(s)           |
|                             | MTMC.OpenAll Op.                 | 0.268(ms)        | 0.698(ms)        | 1.682(ms)         | 3.935(ms)         | 8.841(ms)         | 19.334(ms)         | 42.891(ms)         | 94.463(ms)         |
|                             | $lvc_i$ Generation               | 0.178(ms)        | 0.362(ms)        | 0.643(ms)         | 1.156(ms)         | 2.277(ms)         | 4.316(ms)          | 8.603(ms)          | 17.346(ms)         |
| Proof Update                | MTMC.ComUpdate Op.               | 0.087(ms)        | 0.071(ms)        | 0.078(ms)         | 0.071(ms)         | 0.090(ms)         | 0.076(ms)          | 0.078(ms)          | 0.080(ms)          |
|                             | MTMC.ProofAllUpdate Op.          | 0.001(s)         | 0.010(s)         | 0.020(s)          | 0.770(s)          | 0.792(s)          | 0.831(s)           | 0.902(s)           | 1.033(s)           |
| Proof Aggregation           | MTMC.AggRow Op.                  | 0.025(s)         | 0.073(s)         | 0.171(s)          | 0.483(s)          | 1.059(s)          | 2.270(s)           | 5.080(s)           | 14.621(s)          |
| Proof Verification          | local Index Proof lip Veri.      | 0.004(s)         | 0.010(s)         | 0.021(s)          | 1.671(s)          | 1.870(s)          | 2.041(s)           | 2.252(s)           | 2.470(s)           |
|                             | Aggregated Proof $P_{i,N}$ Veri. | 0.010(s)         | 0.021(s)         | 0.035(s)          | 0.073(s)          | 0.161(s)          | 0.332(s)           | 0.741(s)           | 1.602(s)           |

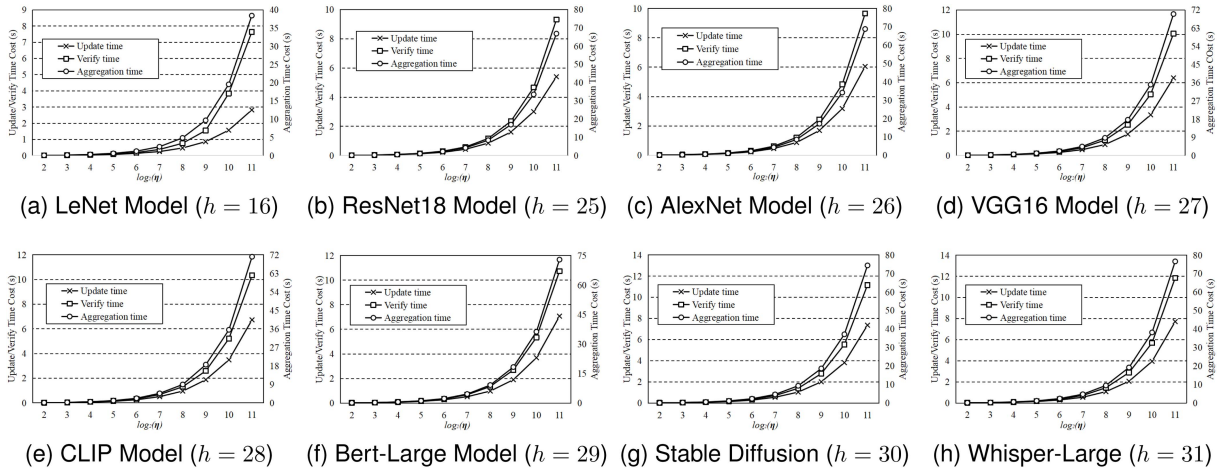


Fig. 5. Time costs of MAMMON scheme for eight ML models. We note that, to more comprehensively demonstrate the performance of MAMMON with respect to update and aggregation, we set the weight vector length  $N = 2^h$  to exceed the actual number of weight parameters in the ML model. Verify time evaluates the time cost of verifying the aggregated proofs.

signature, thereby stealing  $\mathcal{P}$ 's certificate or creating a verifiable certificate  $\text{Cert}(f_{W_P}, \text{MCom}, \text{SIG}_A)$ .

M5:  $\mathcal{A}$  aims to forge an invalid certificate for  $f_{W_P}$ , but it can be verified by  $\mathcal{V}$ . Since the adversary possesses knowledge of the final model parameters,  $\mathcal{A}$  may generate two inconsistent certificates for the same weight matrix.

**Security Properties:** We promise that the scheme should satisfy the following properties:

P1: **Correctness:** If  $\mathcal{P}$  generates the commitment certificate by training from random initialization to convergence at  $f_{W_P}$ , the certificate  $\text{Cert}(f_{W_P}, \text{MCom}, \text{SIG}_P)$  can be successfully verified. Assuming both parties are honest, if  $\mathcal{P}$  correctly follows the commitment protocol during training,  $\mathcal{V}$  will accept MCom.

P2: **Soundness:** It ensures that the model owner rejects any commitment certificate from a dishonest or malicious client. Specifically,  $\mathcal{A}$  can neither impersonate an honest  $\mathcal{P}$ , nor deceive  $\mathcal{V}$  using invalid certificates.

P3: **Aggregability:** By aggregating multiple proofs into a single concise proof,  $\mathcal{P}$  significantly reduces the verification overhead for  $\mathcal{V}$ , which effectively supports scalable multi-client ownership verification for DML models.

P4: **Maintainability:** Upon modification of the model weight vector, the scheme avoids linear-cost recomputation by enabling the update of all associated proofs in sub-linear time. This property ensures the limited overhead for both commitment generation and updates remains.

P5: **Homomorphism:** Homomorphism in matrix commitment allows operations on commitments to yield results equivalent to operating on the underlying vectors, without revealing them. In DML, commitment homomorphism enables privacy-preserving weight aggregation.

### C. Client Training Record Design

CTR, as the minimal data unit to verify the DML ownership of clients, denotes the model intermediate weight checkpoints. The definition of CTR is shown as Definition III.1.

**Definition III.1:** (Client Training Record). For a client  $\mathcal{P}$ , a valid Client Training Record (CTR) is defined as  $\text{CTR} = (\text{CP}, \text{RPK}, \text{Cert}(f_{W_P}, \text{MCom}, \text{SIG}_P), \text{TS}_{\text{SWCD}}, \text{TS}_{\text{RIIW}}, \text{TS}_{\text{MWCD}}, \text{AUX})$ . Specifically, CTR has the following attributes: (a) CP denotes a commitment proof that commits to all attributes of CTR without information disclosure; (b) RPK denotes the public key address which represents the owner of the CTR;

(c)  $\text{Cert}(f_{W_P}, \text{MCom}, \text{SIG}_P)$  denotes the weight checkpoint commitment certificate containing the matrix commitment  $\text{MCom}$  of  $\mathbf{W}$  and the corresponding signature  $\text{SIG}_P$  from  $\mathcal{P}$ ; (d)  $\text{TS}_{SWCD}$  denotes the training specification for the similarity of weight checkpoint distribution; (e)  $\text{TS}_{RIIW}$  denotes the training specification for the distance between initial weight and final weight; (f)  $\text{TS}_{MWCD}$  denotes the training specification for the monotonicity of weight checkpoint distance. (g) the training-specific auxiliary information, denoted by  $\text{AUX}$ , includes the hyperparameters, bias value, etc.

The construction of CTR is shown in Fig. 2. In each global epoch,  $\mathcal{P}$  receives the DML model distributed by  $\mathcal{V}$  and executes local ML computations. The life-cycle of CTR begins when  $\mathcal{P}$  completes the ML training of the first local epoch and initializes the commitment certificate. Within the local epoch,  $\mathcal{P}$  iteratively updates the weight matrix commitment  $\text{MCom}$  at a sub-linear cost. Moreover, the commitment proof  $\text{CP}$  securely binds all properties of CTR, which forms the transaction digest on distributed ledger. Specifically, to reduce ledger overhead while preserving auditability, we batch multiple  $\text{CP}$  update digests into a Merkle tree and record only the Merkle root as a ledger entry (e.g., as a transaction payload). Each leaf corresponds to one checkpoint update, and an inclusion proof can be provided to show that a claimed  $\text{CP}$  is contained in the append-only ledger history.

#### D. Training Specification Design

The training specifications, governing the updates policies of the CTRs, ensure the correct execution of ML model. After several iterations of local training, the client  $\mathcal{P}$  updates the weight checkpoint values in  $\mathbf{W}$ . Once all TSs of the CTR are satisfied, the corresponding  $\text{MCom}$  would be updated alongside the modifications to the matrix  $\mathbf{W}$ .

$\text{TS}_{SWCD}$  (*Similarity of Weight Checkpoint Distribution*): Assuming a small learning rate, the variation between consecutive weight checkpoints  $\mathbf{W}[t-1]$  and  $\mathbf{W}[t]$  should remain bounded. We quantify this using the projection distance (DP) between the low-dimensional projections (LDP) of the  $i$ -th layer weights ( $\mathbf{w}_i$ ) as follows:

$$\text{DP}(\text{LDP}(\mathbf{w}_i[t]), \text{LDP}(\mathbf{w}_i[t-1])) < \delta, \forall i \in [L] \quad (1)$$

Specifically, for a linear layer with weights  $\mathbf{w}_i$  and bias  $\mathbf{b}_i$ , utilizing Euclidean distance instantiates (1) as:

$$\sqrt{(\mathbf{b}_i[t] - \mathbf{b}_i[t-1])^2 + (\|\mathbf{w}_i[t]\|_2^2 - \|\mathbf{w}_i[t-1]\|_2^2)^2} < \delta$$

$\text{TS}_{SWCD}$  satisfies if the adjacent checkpoint pair is  $\delta$ -similar.

$\text{TS}_{RIIW}$  (*Random and Independent Initialization of Weights*): To counteract the potential manipulation by  $\mathcal{A}$  intended to position the initial weight checkpoint  $\mathbf{W}[0]'$  in close proximity to the target weights  $\mathbf{W}[P]$  of the adversarial model  $f_{W_P}^a$ ,  $\text{TS}_{RIIW}$  validates the mutual independence of weights and verifies their initialization via a specific *Gaussian Mixture Model* (GMM). Since weight distributions evolve smoothly as gradients diminish [27], adopting a GMM initialization enhances the salience of the distributional trajectory. Consequently,  $\text{TS}_{RIIW}$  captures deviations from the expected random initialization,

defined by (2), where  $i, j \in [L]$  and  $\text{COV}(\cdot, \cdot)$  denotes the covariance between weight parameters.

$$\begin{cases} \text{DP}(\text{LDP}(\mathbf{w}_i[0]), \text{LDP}_{GMM}(\mathbf{w}_i[0])) \approx 0 \\ \text{COV}(\mathbf{w}_i[0], \mathbf{w}_j[0]) = 0 \end{cases} \quad (2)$$

$\text{TS}_{MWCD}$  (**Monotonicity of Weight Checkpoint Distance**). The normalized distance between weight checkpoints is defined by (3).

$$DL(\mathbf{W}[m], \mathbf{W}[t]) = \frac{\sqrt{\sum_{i=0}^L \|\mathbf{w}_i[m] - \mathbf{w}_i[t]\|_F^2}}{\text{Total}_{wp}} \quad (3)$$

Using the convergence property of SGD, the optimization trajectory is expected to progressively contract towards the final state  $\mathbf{W}[P]$ . Consequently, the distance from an earlier checkpoint to the final model should exceed that of a later checkpoint.  $\text{TS}_{MWCD}$  verifies this monotonic convergence trend, formulated as (4) for any  $m < t$ :

$$\Pr[DL(\mathbf{W}[m], \mathbf{W}[P]) - DL(\mathbf{W}[t], \mathbf{W}[P]) \leq 0] = 0 \quad (4)$$

#### E. Multi-Client Ownership for DML Model

Since CTR involves intermediate weight checkpoints of the client's local ML model, the final ownership of DML model is a sequence of CTRs from multiple provers, refer to Algorithm 1. For **Parameter Initialization**, public parameters  $\text{pp}_K, \text{pp}_{TC}$  and trapdoor information are generated for all other algorithms. Then for **Key Pair Generation**,  $\mathcal{P}$  computes a key pair ( $\text{RPK}, \text{RSK}$ ). Finally in **CTR Generation and Verification**,  $\mathcal{P}$  utilizes parameters such as TSs, public key address, and commitment randomness to compute a trapdoor commitment  $\text{CP}$  for each attributes in a CTR. Specifically, before any commitment ( $\text{MCom}$ ) is updated, the current checkpoint should satisfy all TSs that certify it is obtained by a correct local training step. The commitment  $\text{CP}$  simply denotes an index of the CTR in transactions stored on the distributed ledger.

### IV. MULTI-LINEAR TREE-BASED MATRIX COMMITMENT SCHEME FOR DML MODEL

In the matrix commitment scheme proposed in *Matproof* [29], the entries of an  $L \times N$  matrix  $\mathbf{W}$  are represented as bivariate polynomials. Our matrix commitment scheme adopts such bivariate polynomial function of  $\mathbf{W}$ , denotes as  $\Psi(\mathbf{Y}, \mathbf{R}) = \mathbf{R} \cdot \mathbf{W} \cdot \mathbf{Y}^\top$ , where  $\mathbf{R} = [r_1, \dots, r_L]$  and  $\mathbf{Y} = [y_1, \dots, y_N]$ . The value of polynomial function  $\Psi(\mathbf{Y}, \mathbf{R})$  is mapping into a commitment element in a bilinear group  $\mathbb{G}_1$ . To open a commitment for a specific weight parameter  $w_{i,n}$  at position  $(i, n) \in [L] \times [N]$ , a two-step proof  $P_{i,n}((i, n) \in [L] \times [N])$  is computed, consisting of three items  $(\text{lv}c_i, \text{gip}_i, \text{lip}_{i,n})_{i \in [L], n \in [N]}$  as follows:

- *Local vector commitment* ( $\text{lv}c_i$ ): This represents the commitment to the  $i$ -th row vector  $\mathbf{w}_i$  containing the target element  $w_{i,n}$ . It enables the verification of the weight value at position  $(i, n)$ . Notably, the matrix commitment  $\text{MCom}$  is defined as a commitment to the vector of these local commitments  $[\text{lv}c_1, \dots, \text{lv}c_L]$ .



---

**Algorithm 1: Multi-client Ownership Verification of DML Model.**


---

**Require:** a new CTR for  $\mathcal{P}$  after local ML training

**Ensure:** security parameter  $1^\lambda$ ,

- the weight checkpoint matrix  $\mathbf{W} = [\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_L]^\top$  of an  $L$ -layer ML model,  
 a instance  $\mathbf{w}_i = \{w_{i,n}\}_{n=0}^N = \{w_{i,1}, w_{i,2}, \dots, w_{i,N}\}$ ,  
 training specifications  $\text{TS}_{SWCD}$ ,  $\text{TS}_{RIIW}$ ,  $\text{TS}_{MWCD}$ ,  
 1: */\* Phase 1 : Parameter Initialization for CTR \*/*  
 2:  $\text{pp}_K \leftarrow \text{CTR.KeyPPGen}(1^\lambda)$ : Inputs the security parameter  $\lambda$  and outputs the public parameters  $\text{pp}_K$ .  
 3:  $(\text{pp}_{\text{TC}}, \text{tm}_{\text{TC}}) \leftarrow \text{TCS.TrapGen}(1^\lambda)$ : Inputs  $\lambda$  and outputs  $\text{pp}_{\text{TC}}$  and  $\text{tm}_{\text{TC}}$  for trapdoor commitment.  
 4: */\* Phase 2 : Key Pair Generation for CTR \*/*  
 5: Sample secret key  $sk_{PR}$  for pseudorandom function.  
 6: Sample randomness  $cr_{kp}$  for commitment scheme.  
 7:  $(\text{RPK}, \text{RSK}) \leftarrow \text{CTR.KeyGen}_{\text{pp}_K}(sk_{PR}; cr_{kp})$ : Inputs  $\text{pp}_K$ ,  $sk_{PR}$  and  $cr_{kp}$ , and outputs a key pair (RPK, RSK). The private key (RSK) can watermark matrix commitment.  
 8: */\* Phase 3 : CTR Generation and Verification \*/*  
 9: Generate the  $\text{Cert}(f_{W_P}, \text{MCom}, \text{SIG}_P)$  containing the matrix commitment MCom and  $\mathcal{P}$ 's signature (see Algorithm 2).  
 10:  $0/1 \leftarrow \text{TS}_{SWCD} \wedge \text{TS}_{RIIW} \wedge \text{TS}_{MWCD}$ : Verify the satisfaction of all TSs over each checkpoints and generate a binary value as output. The update execution can only proceed if the verification for all TSs is successful.  
 11:  $\text{CP} \leftarrow \text{TCS.Commit}_{\text{pp}_{\text{TC}}}(\text{RPK} \parallel \text{Cert}(f_{W_P}, \text{MCom}, \text{SIG}_P) \parallel \text{TS}_{SWCD} \parallel \text{TS}_{RIIW} \parallel \text{TS}_{MWCD}; cr_{kp})$ : generate the proof (CP) that combines all attributes of CTR.  
 12:  $\text{CTR} = (\text{CP}, \text{RPK}, \text{Cert}(f_{W_P}, \text{MCom}, \text{SIG}_P), (\text{TS}_{SWCD}, \text{TS}_{RIIW}, \text{TS}_{MWCD}), \text{AUX}; cr_{kp})$ : It outputs the client training record CTR for  $\mathbf{W}$  of  $\mathcal{P}$ 's local ML training.  
 13: A series of CTRs from all clients denotes the identity of final DML model. Return the CTRs for DML model.  
 14:  $cr'_{kp} \leftarrow \text{TCS.TrapOpen}_{\text{pp}_{\text{TC}}}(\text{tm}_{\text{TC}}, \text{CP}, \text{CTR}; cr_{kp})$ : It inputs CTR information and the randomness ( $cr_{kp}$ ) and outputs a trapdoor randomness ( $cr'_{kp}$ ) for  $\mathcal{V}$ .  
 15:  $\{0, 1\} \leftarrow \text{TCS.Open}_{\text{pp}_{\text{TC}}}(\text{CP}, \text{CTR}; cr'_{kp})$ : It inputs CTR information and the trapdoor randomness ( $cr'_{kp}$ ) and outputs a bit value of 0/1 to denote ACCEPT or REJECT.
- 

- *Global index proof* ( $gip_i$ ): This proof validates the position of  $lvc_i$  within the global matrix commitment MCom. It guarantees that the local commitment corresponds to the correct row index  $i$ , thereby verifying the integrity of the row relative to the entire matrix.
- *Local index proof* ( $lip_{i,n}$ ): This proof attests to the correctness of the weight parameter at column index  $n$  within  $lvc_i$ . Combined with the local vector commitment, it enables the precise verification of the specific weight element's value.

An example of matrix commitment is shown as Fig. 3. We remark that if our matrix commitment scheme remains *Correctness* and *Soundness*, the property *Binding*, *Hiding* and

TABLE IV  
STORAGE COSTS UNDER DIFFERENT MULTILINEAR TREE HEIGHTS ( $h$ ), INCLUDING COMMITMENT AND PROOF SIZES, AND KEY MATERIAL SIZES FOR LOCAL VECTOR COMMITMENT ( $lvc$ ). THE *UNI. KEY MAT.* DENOTES UNIVERSAL KEY MATERIAL.

| Storage Cost           | $h=26$ | $h=27$ | $h=28$ | $h=29$ | $h=30$ |
|------------------------|--------|--------|--------|--------|--------|
| MCom (KiB)             | 0.047  | 0.047  | 0.047  | 0.047  | 0.047  |
| $lvc$ (KiB)            | 0.047  | 0.047  | 0.047  | 0.047  | 0.047  |
| $gip$ (KiB)            | 0.047  | 0.047  | 0.047  | 0.047  | 0.047  |
| $lip$ (KiB)            | 1.219  | 1.266  | 1.312  | 1.359  | 1.406  |
| univ. key mat. (MiB)   | 216    | 216    | 216    | 216    | 216    |
| verification key (KiB) | 7.312  | 7.594  | 7.875  | 8.156  | 8.437  |

*Position Binding* holds for matrix commitment. Such commitment scheme offers maintainability and efficient updating of the proof elements when the matrix elements are modified.

#### A. Multi-Linear Tree-Based Proof Generation for MTMC

Our proposed *Multi-linear Tree-based Matrix Commitment* (MTMC) scheme involves four main phases, which are **Proof Generation**, **Proof Update**, **Proof Aggregation** and **Proof Verification**. In MAMMON,  $\mathcal{P}$  computes the commitment MCom and proof  $P_{i,n} = (lvc_i, gip_i, lip_{i,n})_{i \in [L], n \in [N]}$  of the model weight matrix  $\mathbf{W}$ , while  $\mathcal{V}$  verifies the proofs of the commitment. Algorithm 2 shows the structure of our matrix commitment scheme.

For the **Proof Generation for MTMC** phase, we first generate the matrix commitment MCom via the bivariate polynomial function  $\Psi(\mathbf{Y}, \mathbf{R})$ . To watermark the commitment, the proposed MTMC scheme generates two public parameters. On the one hand, the  $\text{MTMC.Gen}(\cdot)$  algorithm generates the initial public parameter  $\text{pp}_{\text{MC}}$  based on the security parameter and matrix dimensions. In particular,  $\text{pp}_{\text{MC}}$  is accessible to all other algorithms in the scheme. On the other hand, the watermarked public parameter  $\text{pp}_{\text{WMC}}$  is generated by  $\text{MTMC.MarkPPGen}(\cdot)$ , allowing for watermarking the Open operations within the matrix commitment scheme. Specifically, we assume  $\mathbf{\Gamma} = [\gamma, \gamma^2, \dots, \gamma^L]$  and  $\mathbf{\Omega} = [\omega, \omega^2, \dots, \omega^N]$  where  $\gamma, \omega \in \mathbb{Z}_p$ . For any value of function  $\Psi(\mathbf{Y}, \mathbf{R})$  at point  $(\gamma, \omega)$ , the matrix commitment MCom of  $\mathbf{W}$  is computed as  $g_1^{\Psi(\gamma, \omega)}$ . Hence, the  $\text{MTMC.Com}(\cdot)$  computes the matrix commitment as:  $\text{MCom} = g_1^{\Psi(\mathbf{\Gamma}, \mathbf{\Omega})} = g_1^{\mathbf{\Omega} \cdot \mathbf{W} \cdot \mathbf{\Gamma}^\top}$ . Additionally, the  $\text{MTMC.Open}(\cdot)$  algorithm computes an individual proof  $P_{i,n} = (lvc_i, gip_i, lip_{i,n})_{i \in [L], n \in [N]}$ . To ensure the *Position Binding* of  $lvc_i$  with MCom, the global index proof is quantified as:  $gip_i = g_1^{\omega^{L+1-i} \mathbf{\Omega}^\top [-i] (\mathbf{W} \cdot \mathbf{\Gamma}) [-i]}$ .

Inspired by *Hyperproof* [30] and *PST* [31], we computes the local vector commitment by extending the vector into a multi-linear tree. Such multi-linear tree extension supports the *homomorphism* of commitment, which ensure the computational integrity of global DML model aggregation. Without loss of generality, we assume that the number of columns of matrix  $\mathbf{W}$  is  $N = 2^h$ . The vector  $\mathbf{w}_i = [w_{i,1}, w_{i,2}, \dots, w_{i,N}]$  at the  $i$ -th row of  $\mathbf{W}$  can be multi-linearly extended to a polynomial function  $\tilde{w} : \mathbb{Z}_p^h \rightarrow \mathbb{Z}_p$ . The multi-linear extension polynomial  $\tilde{w}$  maps  $n$ -th position of  $\mathbf{w}_i$  to  $w_{i,n}$  is shown as:  $\tilde{w}(\mathbf{n}) = \tilde{w}(n_h, n_{h-1}, \dots, n_1) = w_{i,n}$ . In order to extend  $\tilde{w}$  from  $w_{i,n}$  to  $\mathbf{w}_i$ , we assume a trapdoor  $\mathbf{a} = (a_h, a_{h-1}, \dots, a_1) \in \mathbb{R} \mathbb{Z}_p^h$

---

**Algorithm 2:** Multi-linear Tree-based Matrix Commitment Scheme.

---

**Require:** the commitment MCom and the proof items  $(lvc_i, gip_i, lip_{i,n})_{i \in [L], n \in [N]}$  of the  $\mathbf{W}$

**Ensure:** security parameter  $1^\lambda$ , the model weight matrix  $\mathbf{W} = [\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_L]^\top$ , the number  $L$  of  $\mathbf{W}$  rows, the number  $N$  of  $\mathbf{W}$  columns, the key pair (RPK, RSK) of client  $\mathcal{P}$

- 1: /\* Phase 1 : **Proof Generation** for MTMC \*/
- 2:  $\text{pp}_{\text{MTC}} \leftarrow \text{MTMC.Gen}(1^\lambda, 1^L, 1^N)$ : Inputs  $\lambda$ , row numbers ( $L$ ) and column numbers ( $N$ ) of  $\mathbf{W}$ , and outputs  $\text{pp}_{\text{MTC}}$ .
- 3:  $\text{pp}_{\text{WMC}} \leftarrow \text{MTMC.MarkPPGen}(\text{pp}_{\text{MTC}}, \text{RSK})$ : Inputs  $\text{pp}_{\text{MTC}}$  and RSK, and outputs a watermarked public parameters  $\text{pp}_{\text{WMC}}$ .
- 4:  $\text{MCom} \leftarrow \text{MTMC.Com}_{\text{pp}_{\text{WMC}}}(\mathbf{W})$ : Inputs the weight checkpoint matrix  $\mathbf{W}$  and outputs the commitment MCom.
- 5:  $P_{i,n} \leftarrow \text{MTMC.Open}_{\text{pp}_{\text{WMC}}}(w_{i,n}, i, n)$ : Inputs the specific position  $(i, n)_{i \in [L], n \in [N]}$  and the message  $w_{i,n}$  at the  $(i, n)$ -th position in  $\mathbf{W}$ . It computes the proof  $P_{i,n}$  by the watermarked public parameters  $\text{pp}_{\text{WMC}}$  under RSK.
- 6:  $[P_{i,1}, P_{i,2}, \dots, P_{i,N}] \leftarrow \text{MTMC.OpenAll}_{\text{pp}_{\text{WMC}}}(\mathbf{w}_i, i)$ : Inputs the row  $i$  and outputs all proof  $P_{i,n}$  for  $\mathbf{w}_i$ .
- 7: /\* Phase 2 : **Proof Update** for MTMC \*/
- 8:  $\text{MCom}' \leftarrow \text{MTMC.ComUpdate}_{\text{pp}_{\text{WMC}}}(\text{MCom}, i, n, \xi)$  ( $\xi \in \mathbb{Z}_p$ ): Inputs the position  $(i, n) \in [L] \times [N]$  and MCom, and outputs the updated MCom' computed by the  $(i, n)$ -th position with  $\xi$ .  $\xi$  represents the change value of  $w_{i,n}$ .
- 9:  $P'_{i',n'} \leftarrow \text{MTMC.ProofAllUpdate}_{\text{pp}_{\text{WMC}}}(i', n', \xi, P_{i',n'})$  ( $\xi \in \mathbb{Z}_p$ ): Once the element  $w_{i,n}$  is updated into  $w_{i,n} + \xi$ , it outputs every updated local index proof  $P'_{i',n'}$  computed by the the  $(i', n')$ -th position with change value  $\xi$ .
- 10: /\* Phase 3 : **Proof Aggregation** for MTMC \*/
- 11:  $P_{i,N} \leftarrow \text{MTMC.AggRow}_{\text{pp}_{\text{WMC}}}([w_{i,n}]_{n \in N}, [P_{i,n}]_{n \in N}, i, N)$ : Inputs all individual parameters  $[w_{i,n}]_{n \in N}$  and all proofs  $[P_{i,n}]_{n \in N}$  of the  $i$ -th row in  $\mathbf{W}$ . It aggregates them into one proof  $P_{i,N}$ .
- 12: /\* Phase 4 : **Proof Verification** for MTMC \*/
- 13:  $\{0, 1\} \leftarrow \text{MTMC.Verify}_{\text{pp}_{\text{WMC}}}(\text{MCom}, i, N, P_{i,N}, w_{i,n}, \text{RPK})$  ( $(i, n) \in [L] \times [N]$ ): Inputs  $P_{i,N}$ , MCom and  $w_{i,n}$ . It outputs 1 to denote that  $w_{i,n}$  is the  $(i, n)$ -th message of  $\mathbf{W}$ . Otherwise, it outputs 0.  $\mathcal{V}$  verifies the watermarks of the proof  $P_{i,N}$  via RPK.

---

and define the selection functions  $S_{k,j}$  and  $\Upsilon_{k,h}$  ( $k \in [0, 2^h)$ ) as follows:

$$S_{k,j}(a_j) = \begin{cases} a_j, & k_j = 1 \\ 1 - a_j, & k_j = 0 \end{cases}, \Upsilon_{k,h}(\mathbf{a}) = \prod_{j=1}^h S_{k_j,h}(a_j) \quad (5)$$

We note that  $\Upsilon_{0,0}(\mathbf{a}) = 1$ . The multi-linear extended polynomial function  $\tilde{w}$  of  $\mathbf{w}_i$  is as:

$$\begin{aligned} \tilde{w}(\mathbf{a}) &= \sum_{k=0}^{N-1} w_{i,k+1} \Upsilon_{i,k,h}(a_h, a_{h-1}, \dots, a_1) \\ &= \sum_{k=0}^{N-1} w_{i,k+1} \Upsilon_{i,k,h}(\mathbf{a}) \end{aligned}$$

We remark that the element  $i$  for  $\Upsilon_{i,k,h}$  denotes the index of the weight vector  $\mathbf{w}_i$ .  $\Upsilon_{i,k,h}$  denotes the computation of  $\Upsilon_{k,h}$  on the element  $w_{i,k}$  in the vector  $\mathbf{w}_i$ .

The MTMC generates the local vector commitment  $lvc$  via a pairing tuple  $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, e)$  and  $g_2^{t_j - n_j}$  ( $j \in [h]$ ) is

---

**Algorithm 3:** Multi-linear Tree-based Local Vector Commitment Generation.

---

**Require:** multi-linear extended polynomial  $\tilde{w}$ , and the number  $h$ ,

- 1: **if**  $N \neq 0$  **then**
- 2:  $(\tilde{w}_1 - \tilde{w}_0, \tilde{w}_t) \leftarrow \text{Dec}(\tilde{w}, t)$ : for  $\forall t \in \{0, 1\}$ , the decomposition of  $\tilde{w}$  involves quotient  $\tilde{w}_1 - \tilde{w}_0$  and remainder  $\tilde{w}_t$  by  $a_h - t$  such that  $\tilde{w} = (\tilde{w}_1 - \tilde{w}_0) \cdot (a_h - t) + \tilde{w}_t$ .
- 3: **Return**  $(g_1^{(\tilde{w}_1 - \tilde{w}_0)(\mathbf{a})}, \text{Dec}(\tilde{w}_0, h-1), \text{Dec}(\tilde{w}_1, h-1))$
- 4: **else**
- 5: the  $\tilde{w}$  is a constant and outputs  $\phi$ .
- 6: **end if**

---

the verification key of the  $n$ -th position in  $\mathbf{w}_i$ . (6) as follows is the local vector commitment  $lvc$  computed via  $g_1$ ,  $\mathbf{w}_i$  and  $\mathbf{a}$  (line 5 in Algorithm 2):

$$lvc = g_1^{\tilde{w}(\mathbf{a})} = g_1^{\tilde{w}(a_h, a_{h-1}, \dots, a_1)} = \prod_{k=0}^{h-1} \left( g_1^{\Upsilon_{i,k,h}(\mathbf{a})} \right)^{w_{i,k+1}} \quad (6)$$

We reduce the  $\text{MTMC.Open}(\cdot)$  time costs to  $O(N \log N)$  via a multi-linear tree for  $\mathbf{w}_i$  of length  $N$ . The multi-linear tree structure is generated by the decomposition of the multi-linear extended polynomial function  $\tilde{w}$  as shown in Algorithm 3. First, we divide the polynomial function  $\tilde{w}$  by the term  $a_h - t$  ( $t \in \{0, 1\}$ ), resulting in two multi-linear extended polynomial functions, denoted as  $\tilde{w}_0$  and  $\tilde{w}_1$ . We express  $\tilde{w}$  as the linear combination  $(\tilde{w}_1 - \tilde{w}_0) \cdot (a_h - t) + \tilde{w}_t$  (line 2 in Algorithm 3). For example, we assume a vector  $\mathbf{v} = [v_1, v_2, v_3, v_4]$  of size  $N = 4, h = 2$ , and  $\mathbf{v}_0 = [v_1, v_2]$  and  $\mathbf{v}_1 = [v_3, v_4]$  denote the left and right halves of vector  $\mathbf{v}$ . The vectors  $\mathbf{v}_0$  and  $\mathbf{v}_1$  can be multi-linear extended into polynomial functions  $\tilde{w}_0$  and  $\tilde{w}_1$ . We quantify the multi-linear tree decomposition of  $\tilde{w}$  as:  $\tilde{w} = (1 - a_h) \cdot \tilde{w}_0 + a_h \cdot \tilde{w}_1 = (\tilde{w}_1 - \tilde{w}_0) \cdot (a_h - 1) + \tilde{w}_1$ . Next, we generate a commitment  $g_1^{(\tilde{w}_1 - \tilde{w}_0)(\mathbf{a})}$  via (6). Finally, we recursively disperse the polynomial functions  $\tilde{w}_0$  and  $\tilde{w}_1$  into two new polynomial functions, repeating the process until  $\tilde{w}$  becomes a constant, i.e., further divisions are not possible.

Such recursive process creates a multi-linear tree structure in our local vector commitment scheme, as shown in Fig. 4. The set of local index proof  $(lip_{i,1}, lip_{i,2}, \dots, lip_{i,N})$  can be represented by the multi-linear tree (line 5 in Algorithm 2). Specifically, Each  $lip_{i,n}$  corresponds to the  $n$ -th path in the multi-linear tree of  $\mathbf{w}_i$  and can be denoted as  $lip_{i,n} = (x_{i,n,b}, x_{i,n,b-1}, \dots, x_{i,n,1})$  (line 6 in Algorithm 2). Here,  $x_{i,n,b}$  represents the  $b$ -th node of the  $n$ -th path in the multi-linear tree of  $\mathbf{w}_i$ . After performing the necessary computations, the individual proof  $P_{i,n} = (lvc_i, gip_i, lip_{i,n})_{i \in [L], n \in [N]}$  is generated. Multi-linear tree is a divide-and-conquer algorithm that decomposes a polynomial into smaller-scale sub-problems and leverages the relationships between these sub-problems to perform recursive computations. Such process ultimately transforms the polynomial from its coefficient representation to a point-valued representation. The time complexity of such recursive computation is quantified as  $T(h) = 2T(h/2)$ . Applying the *Master Theorem* to the equation



TABLE V  
SHA256, PEDERSEN, POSEIDON SCHEME VERSUS MTMC SCHEME WITH  
FIXED  $\eta = 16$

| Time Costs                                 | Scheme   | $\eta = 16$  |              |              |              |
|--------------------------------------------|----------|--------------|--------------|--------------|--------------|
|                                            |          | $N = 8$      | $N = 16$     | $N = 32$     | $N = 64$     |
| <b>Parameter Generation</b> (s)            | Sha256   | 33.574       | 37.773       | 52.396       | 57.182       |
|                                            | Pedersen | 2.112        | 2.980        | 3.228        | 3.984        |
|                                            | Poseidon | 0.257        | 0.305        | 0.340        | 0.510        |
|                                            | MTMC     | <b>0.014</b> | <b>0.044</b> | <b>0.153</b> | 0.621        |
| <b>Commitment and Proof Generation</b> (s) | Sha256   | 182.324      | 186.124      | 356.360      | 359.823      |
|                                            | Pedersen | 22.988       | 36.390       | 38.016       | 39.852       |
|                                            | Poseidon | 6.227        | 7.272        | 8.077        | 10.417       |
|                                            | MTMC     | <b>0.006</b> | <b>0.020</b> | <b>0.076</b> | <b>0.291</b> |
| <b>Proof Verification</b> (s)              | Sha256   | 0.003        | 0.003        | 0.003        | 0.003        |
|                                            | Pedersen | 0.003        | 0.003        | 0.003        | 0.003        |
|                                            | Poseidon | 0.003        | 0.003        | 0.003        | 0.003        |
|                                            | MTMC     | 0.010        | 0.021        | 1.671        | 1.870        |

above, we can conclude that the overall time complexity of the MTMC.Open( $\cdot$ ) algorithm is  $O(N \log N)$ .

### B. Commitment and Proof Update for MTMC

For **Proof Update** phase, we assume that the weight value  $w_{i,n}$  (at  $(i,n)$ -th position in  $\mathbf{W}$ ) changes by  $\xi$  after a few local training epochs (**P4**). The MTMC.ComUpdate( $\cdot$ ) algorithm updates the new matrix commitment as:  $\text{MCom}' = \text{MCom} \cdot g_1^{\xi \cdot \gamma^n \cdot \omega^i}$ . Moreover, for MTMC.ProofAllUpdate( $\cdot$ ), some auxiliary information  $aux$  for position  $(i,n)$  is required to update the local vector commitment  $lvc$  [32]. Refer to (5), The  $aux_{i,n}$  for position  $(i,n)$  is as:  $aux_{i,n} = \{aux_{i,n,b}, b \in [0, h]\} = \{g_1^{\Upsilon_{i,n-1,b}(\mathbf{a})}, b \in [0, h]\}$ . Thus for any  $b \in [0, h]$ ,  $lip_{i,n} = (x_{i,n,h}, x_{i,n,h-1}, \dots, x_{i,n,1})$  of the  $(i,n)$ -th path and the local vector commitment  $lvc'_i$  can be updated as (7) and (8) (line 9 in Algorithm 2):

$$x'_{i,n,b} = x_{i,n,b} \cdot (aux_{i,n,b-1})^\xi = x_{i,n,b} \cdot (g_1^{\Upsilon_{i,n-1,b-1}(\mathbf{a})})^\xi \quad (7)$$

$$lvc'_i = lvc_i \cdot g_1^{\Upsilon_{i,n-1,h}(\mathbf{a})} = lvc_i \cdot (aux_{i,n,h})^\xi \quad (8)$$

Such case of  $i' = i, n' = n$ , the MTMC.ProofAllUpdate( $\cdot$ ) algorithm outputs the  $P'_{i,n} = (lvc'_i, gip_i, lip'_{i,n})$ .

As for the proof  $lip_{i,n'}$  of  $w_{i,n'}$  at position  $n \neq n'$ , we consider the same nodes where the  $n'$ -th path and the  $n$ -th path intersect on the multi-linear tree of  $\mathbf{w}_i$ . Specifically, we assume that  $n$  and  $n'$  have  $j$  same bits, which means for any  $b \in [h-j+1, h]$ ,  $i_b = i'_b$ . Thus the proof  $lip_{i,n'}$  of  $w_{i,n'}$  can be updated as:  $x'_{i,n',b} = x_{i,n',b} \cdot (aux_{i,n',b-1})^\xi = x_{i,n',b} \cdot (g_1^{\Upsilon_{i,n'-1,b-1}(\mathbf{a})})^\xi$ . The local vector commitment  $lvc$  is updated as:  $lvc'_i = lvc_i \cdot g_1^{\Upsilon_{i,n'-1,h}(\mathbf{a})} = lvc_i \cdot (aux_{i,n',h})^\xi$ . Such case of  $i' = i, n' \neq n$ , the MTMC.ProofAllUpdate( $\cdot$ ) algorithm outputs the  $P'_{i,n'} = (lvc'_i, gip_i, lip'_{i,n'})$ . In addition, as for the global index proof  $gip_{i',n'}$  of  $w_{i',n'}$  at position  $i' \neq i$ , we quantify the global index proof  $gip'_{i'}$  as:  $gip'_{i'} = gip_i \cdot g_1^{\xi \gamma^n \omega^{L+1-i'+i}}$ . Such case of  $i' \neq i$ , the MTMC.ProofAllUpdate( $\cdot$ ) algorithm outputs the  $P'_{i',n'} = (lvc'_{i'}, gip'_{i'}, lip'_{i',n'})$ . Due to the multi-tree structure, the effective updates of proof  $lip_{i,n}$  involves  $O(\log N)$  leaf nodes for the  $i$ -th path for the tree. Hence, the proof updates time cost for the weight vector  $\mathbf{w}_i$  is  $O(\log N)$ .

### C. Proof Aggregation and Verification for MTMC

**Proof Aggregation:** our scheme uses *non-interactive Inner Product Arguments* (IPA) [30] to aggregate proofs (**P3**). To aggregate a series of individual proofs  $P_{i,n}$  into a proof  $P_{i,N}$ , we assume that vector  $\mathbf{R} = [\mathbf{R}_1 || \mathbf{R}_2 || \dots || \mathbf{R}_\eta]_{\eta \in [N]}$  denotes proofs  $P_{i,n} (n \in N)$  and  $\mathbf{T} = [\mathbf{T}_1 || \mathbf{T}_2 || \dots || \mathbf{T}_\eta]_{\eta \in [N]}$  denotes the corresponding verification keys, where  $\mathbf{R}, \mathbf{T} \in \mathbb{G}_1^\eta \times \mathbb{G}_2^\eta$ . We define the MTMC.AggRow algorithm as follows:

MTMC.AggRow<sub>ppmc</sub> ( $[w_{i,n}]_{n \in N}, [P_{i,n}]_{n \in N}, i, N$ ):

- 1) The pairing product  $\langle \mathbf{R}, \mathbf{T} \rangle = \prod_{j=1}^\eta e(\mathbf{R}_j, \mathbf{T}_j)$ .
- 2) A commitment key pairing  $(\text{RPK}, \text{RSK}) \in \mathbb{G}_1^\eta \times \mathbb{G}_2^\eta$  used to compute  $CB \leftarrow \langle \mathbf{R}, \text{RSK} \rangle$ .
- 3)  $v_j \leftarrow H(CB, \mathbf{T}, j) \in \mathbb{Z}_p$  where  $H(\cdot)$  is a random oracle function and  $j \in [\eta]$ .
- 4) Compute  $\mathbf{T}' \leftarrow [\mathbf{T}_1^{v_1} || \mathbf{T}_2^{v_2} || \dots || \mathbf{T}_\eta^{v_\eta}]$  for  $\eta \in [N]$ .
- 5) Return  $P_{i,N} \leftarrow (CB, (\text{IPA.Compute}(\text{ComKey}, \mathbf{R}, \mathbf{T}')))$ , where the IPA.Compute algorithm is used to compute the aggregated proof  $P_{i,N} \leftarrow \text{IPA.Com}((\text{RPK}, \text{RSK}); \mathbf{R}, \mathbf{T}, \langle \mathbf{R}, \mathbf{T} \rangle)$

For typical verifiable computation, the proof aggregation in SNARK predominantly governed by the operations of multiple multi-exponentiations and Fast Fourier Transforms (FFTs). Therefore, SNARK cannot support effective proof aggregation. Our MTMC scheme demonstrates significantly higher efficiency in aggregation compared to Merkle SNARK. For the same tree structure of depth  $h$ , MTMC with IPA-based prover only requires computing  $O(\eta \cdot h)$  pairings and  $O(\eta \cdot h)$ -sized  $\mathbb{G}_1, \mathbb{G}_2$  and  $\mathbb{G}_T$  exponentiations to aggregate  $N$  proofs.

For **Proof Verification** phase, we assume that  $\theta$  is the private watermarking key (RSK) of  $\mathcal{P}$  and we embed it into the proof  $lip_{i,n}$  as (line 5 in Algorithm 2):  $lip_{i,n} = (x_{i,n,h}^\theta, x_{i,n,h-1}^\theta, \dots, x_{i,n,1}^\theta)$ . The corresponding public verification key (RPK) is  $g_2^\theta$ .  $\mathcal{V}$  verifies the effectiveness of proof  $lip_{i,n}^\theta$  by computing  $e(lvc_i/g_1^{a_n}, g_2^\theta)$  and  $\prod_{j \in [n]} e(x_{i,n,j}^\theta, g_2^{a_j-n_j})$ , respectively (line 13 in Algorithm 1). The MTMC.Verify( $\cdot$ ) algorithm would accept the proof  $P_{i,N}$  in the case where both (9a) and (9b) hold as follows (i.e., the proof  $P_{i,N}$  is effective):

$$e(lvc_i/g_1^{a_n}, g_2^\theta) = \prod_{j \in [n]} e(x_{i,n,j}^\theta, g_2^{a_j-n_j}) \quad (9a)$$

$$e(\text{MCom}, g_2^{\omega^{L+1-i}}) = e(gip_i, g_2) \cdot e(g_1^{\mathbf{w}_i \cdot \mathbf{T}^\top}, g_2^{\omega^{L+1}}) \quad (9b)$$

## V. SECURITY ANALYSIS

### A. Security of CTR and TSs

Refer to the threat model and the two properties of effective proof, we define three threats (**M1**, **M2**, **M3**) by adversaries with capabilities in our threat model. Our following analysis of CTR and TRs shows that none of them can defeat MAMMON under our threat model.

**Security of Three TSs:** We consider an adaptive optimization-based adversary ( $\mathcal{A}$ ) who aims to forge a trajectory  $\bar{\mathbf{W}}_{\text{adv}} =$

TABLE VI

COMPARISON OF  $\text{TS}_{SWCD}$  ( $\text{TS}_S$ ),  $\text{TS}_{RIIW}$  ( $\text{TS}_R$ ), AND  $\text{TS}_{MWCD}$  ( $\text{TS}_M$ ) AFTER LAUNCHING CFA AND REA UNDER VARIOUS MODELS. † DENOTES THAT THE CORRESPONDING TS DETECTS THE ATTACK. ACC. MONO. DENOTES THE ACCURACY MONOTONICITY.

| Dataset   | Model    | Attack | $\text{TS}_S$ | $\text{TS}_R$ (DP) | $\text{TS}_R$ (COV) | $\text{TS}_M$ | Acc. Mono. |
|-----------|----------|--------|---------------|--------------------|---------------------|---------------|------------|
| CIFAR 10  | LeNet    | Ours   | 0.094         | 0.105              | 0.277               | 0.105         | 0.990      |
|           |          | CFA    | 0.003         | 0.013              | 0.260               | 0.119†        | 1.000      |
|           |          | REA    | 0.050         | 0.012              | 0.012               | 0.120†        | 0.980      |
|           | ResNet18 | Ours   | 0.177         | 0.012              | 0.138               | 0.035         | 0.964      |
|           |          | CFA    | 0.004         | 0.005              | 0.135               | 0.035†        | 1.000      |
|           |          | REA    | 0.019         | 0.012              | 0.135               | 0.035†        | 0.894      |
|           | AlexNet  | Ours   | 0.033         | 0.007              | 0.126               | 0.028         | 0.984      |
|           |          | CFA    | 0.004         | 0.009              | 0.124               | 0.032†        | 1.000      |
|           |          | REA    | 0.025         | 0.013              | 0.130               | 0.038†        | 0.964      |
|           | VGG16    | Ours   | 0.030         | 0.011              | 0.141               | 0.025         | 0.986      |
|           |          | CFA    | 0.004         | 0.008              | 0.145               | 0.028†        | 1.000      |
|           |          | REA    | 0.027         | 0.012              | 0.132               | 0.030†        | 1.000      |
| CIFAR 100 | LeNet    | Ours   | 0.079         | 0.011              | 0.246               | 0.124         | 0.991      |
|           |          | CFA    | 0.006         | 0.038†             | 0.275†              | 0.122         | 1.000      |
|           |          | REA    | 0.060         | 0.012              | 0.265†              | 0.129†        | 0.984      |
|           | ResNet18 | Ours   | 0.179         | 0.012              | 0.091               | 0.035         | 0.981      |
|           |          | CFA    | 0.004         | 0.037†             | 0.090               | 0.035         | 1.000      |
|           |          | REA    | 0.061         | 0.013              | 0.266†              | 0.129†        | 0.984      |
|           | AlexNet  | Ours   | 0.035         | 0.012              | 0.092               | 0.029         | 0.985      |
|           |          | CFA    | 0.003         | 0.015              | 0.096               | 0.033†        | 1.000      |
|           |          | REA    | 0.027         | 0.011              | 0.091               | 0.044†        | 0.992      |
|           | VGG16    | Ours   | 0.032         | 0.011              | 0.104               | 0.028         | 0.979      |
|           |          | CFA    | 0.004         | 0.020†             | 0.092               | 0.028†        | 1.000      |
|           |          | REA    | 0.021         | 0.012              | 0.093               | 0.030†        | 0.647      |
| VOC 2012  | UNet     | Ours   | 0.150         | 0.048              | 0.107               | 0.024         | 0.748      |
|           |          | CFA    | 0.010         | 0.049              | 0.118†              | 0.025†        | 1.000      |
|           |          | REA    | 0.051         | 0.048              | 0.110†              | 0.024         | 0.660      |
|           | YOLOv8n  | Ours   | 0.313         | 0.131              | 0.176               | 0.079         | 0.692      |
|           |          | CFA    | 0.074         | 0.120              | 0.182†              | 0.074         | 1.000      |
|           |          | REA    | 0.004         | 0.154†             | 0.180†              | 0.071         | 0.500      |

$\{\tilde{\mathbf{W}}[t]\}_{t=0}^P$  that converges to a known final weight parameters  $\mathbf{W}[P]$ .  $\mathcal{A}$  may use any adaptive, gradient-based, second-order, or combinatorial optimization method, but must output a training history that satisfies all TSs simultaneously. Specifically,  $\mathcal{A}$  attempts to satisfy:

- $\text{TS}_{SWCD}$ : the forged training history must be smooth such that  $\|\tilde{\mathbf{W}}[t] - \tilde{\mathbf{W}}[t-1]\|_2 \leq \delta_{\text{adv}}$  for all  $t \in [P]$ .
- $\text{TS}_{RIIW}$ : the forged initialization  $\tilde{\mathbf{W}}[0]$  must be statistically indistinguishable from the random initialization distribution. For analysis we use the standard proxy  $\tilde{\mathbf{W}}[0] \sim \mathcal{N}(0, \sigma^2 I_d)$  where  $d$  denotes the dimension of the model weight.
- $\text{TS}_{MWCD}$ : the forged checkpoint must converge to  $\mathbf{W}[P]$  with  $\tilde{\mathbf{W}}[P] = \mathbf{W}[P]$ , and moreover its initial-to-converged distance must be compatible with genuine training history:  $\|\tilde{\mathbf{W}}[0] - \mathbf{W}[P]\|_2 \leq DL_{\text{true}} + \tau$  where  $DL_{\text{true}} := \|\mathbf{W}_{\text{real}}[0] - \mathbf{W}[P]\|_2$  is the genuine initial-to-converged distance for a real training producing  $\mathbf{W}[P]$ , and  $\tau \geq 0$  is a verification tolerance.

Let  $\alpha \in (0, 1)$  denote the probability of the random-initialization concentration event fail, and let  $\varepsilon_0 \in (0, 1)$  denote the corresponding relative concentration slack. Motivated by measure concentration in high-dimensional spaces, we have  $\mathbb{E}\|\tilde{\mathbf{W}}[0]\|_2^2 = \sigma^2 d$  for  $\tilde{\mathbf{W}}[0] \sim \mathcal{N}(0, \sigma^2 I_d)$ . Hence, the norm of a  $d$ -dimensional isotropic Gaussian vector concentrates sharply around its expectation  $\mathbb{E}\|\tilde{\mathbf{W}}[0]\|_2 \approx \sigma\sqrt{d}$  (its relative

fluctuations are on the order of  $O(\sigma/\sqrt{d})$ ). Accordingly, we introduce the factor  $(1 - \varepsilon_0)$  to set a lower threshold that accommodates permissible statistical deviations, and define the event as  $\mathcal{E}(\alpha, \varepsilon_0) := \{\|\tilde{\mathbf{W}}[0]\|_2 \geq (1 - \varepsilon_0)\sigma\sqrt{d}\}$ .

**Theorem V.1 (Infeasibility of Optimization-based Forgery):** Let  $\alpha, \varepsilon_0 \in (0, 1)$  be verification parameters such that  $\text{TS}_{RIIW}$  implies the concentration bound  $\Pr[\mathcal{E}(\alpha, \varepsilon_0)] \geq 1 - \alpha$ . Define the deterministic distance lower bound as  $DL_{\min} := (1 - \varepsilon_0)\sigma\sqrt{d} - \|\mathbf{W}[P]\|_2$ . Consider  $\mathcal{A}$  constrained by a smoothness budget  $\delta_{\text{adv}}$  and duration  $P$ . Let  $\text{Succ}_{\text{all}}$  denote the event that the  $\mathcal{A}$  forges checkpoints  $\{\tilde{\mathbf{W}}[t]\}_{t=0}^P$  satisfying  $\text{TS}_{RIIW}$ ,  $\text{TS}_{SWCD}$  and  $\text{TS}_{MWCD}$  simultaneously. If the following condition holds:

$$DL_{\min} > \max\{P \cdot \delta_{\text{adv}}, P \cdot \delta_{\text{real}} + \tau\} \quad (10)$$

then  $\mathcal{A}$ 's success probability is bounded by  $\Pr[\text{Succ}_{\text{all}}] \leq \alpha$ .

*Proof:* We establish contradictions by comparing a high-probability lower bound on  $\|\tilde{\mathbf{W}}[0] - \mathbf{W}[P]\|_2$  forced by  $\text{TS}_{RIIW}$  with upper bounds implied by  $\text{TS}_{SWCD}$  and  $\text{TS}_{MWCD}$ . For the lower bound from  $\text{TS}_{RIIW}$ ,  $\tilde{\mathbf{W}}[0] \sim \mathcal{N}(0, \sigma^2 I_d)$ . By the reverse triangle inequality, we have  $\|\tilde{\mathbf{W}}[0] - \mathbf{W}[P]\|_2 \geq \|\tilde{\mathbf{W}}[0]\|_2 - \|\mathbf{W}[P]\|_2$ . On the event  $\mathcal{E}(\alpha, \varepsilon_0)$ , we have  $\|\tilde{\mathbf{W}}[0]\|_2 \geq (1 - \varepsilon_0)\sigma\sqrt{d}$ , and thus

$$\|\tilde{\mathbf{W}}[0] - \mathbf{W}[P]\|_2 \geq (1 - \varepsilon_0)\sigma\sqrt{d} - \|\mathbf{W}[P]\|_2 = DL_{\min}. \quad (11)$$

Since  $\Pr(\mathcal{E}(\alpha, \varepsilon_0)) \geq 1 - \alpha$ , the (11) holds with probability at least  $1 - \alpha$ .

For upper bound from  $\text{TS}_{SWCD}$ , the step-wise movements yields as follows if  $\tilde{\mathbf{W}}[P] = \mathbf{W}[P]$ :

$$\begin{aligned} \|\tilde{\mathbf{W}}[0] - \mathbf{W}[P]\|_2 &\leq \|\tilde{\mathbf{W}}[0] - \tilde{\mathbf{W}}[P]\|_2 + \|\tilde{\mathbf{W}}[P] - \mathbf{W}[P]\|_2 \\ &\leq \sum_{t=1}^P \|\tilde{\mathbf{W}}[t] - \tilde{\mathbf{W}}[t-1]\|_2 \leq P \cdot \delta_{\text{adv}}. \end{aligned}$$

For a genuine checkpoints under  $\text{TS}_{MWCD}$ , by telescoping and the triangle inequality, we have  $\|\mathbf{W}_{\text{real}}[0] - \mathbf{W}[P]\|_2 \leq \sum_{t=1}^P \|\mathbf{W}_{\text{real}}[t] - \mathbf{W}_{\text{real}}[t-1]\|_2 \leq P \cdot \delta_{\text{real}}$ . Therefore any forged checkpoints passing  $\text{TS}_{MWCD}$  must satisfy

$$\|\tilde{\mathbf{W}}[0] - \mathbf{W}[P]\|_2 \leq P \cdot \delta_{\text{real}} + \tau.$$

Therefore, the distance of any successful forged training history ( $\text{Succ}_{\text{all}}$ ) is bounded by the minimum of these two limits:

$$\|\tilde{\mathbf{W}}[0] - \mathbf{W}[P]\|_2 \leq \min\{P \cdot \delta_{\text{adv}}, P \cdot \delta_{\text{real}} + \tau\}. \quad (12)$$

If (10) holds, (12) contradicts the lower bound (11) on  $\mathcal{E}(\alpha, \varepsilon_0)$ . This implies that on the event  $\mathcal{E}(\alpha, \varepsilon_0)$ , the intersection of the feasible sets for  $\text{TS}_{RIIW}$  and the other specifications is empty. Therefore,  $\Pr[\text{Succ}_{\text{all}} \wedge \mathcal{E}(\alpha, \varepsilon_0)] = 0$ , and  $\Pr[\text{Succ}_{\text{all}}] \leq \Pr[\neg \mathcal{E}(\alpha, \varepsilon_0)] \leq \alpha$ .

To guarantee the random-initialization event  $\mathcal{E}(\alpha, \varepsilon_0)$  holds with probability at least  $1 - \alpha$ , we examine the normalized squared norm  $X := \|\tilde{\mathbf{W}}[0]\|_2^2 / \sigma^2$ , which follows a Chi-squared distribution  $\chi_d^2$ . Applying the Laurent-Massart lower-tail bound  $\Pr[X \leq d - 2\sqrt{dx}] \leq e^{-x}$  with  $x = \ln(1/\alpha)$  implies that  $X \geq$

$d - 2\sqrt{d \ln(1/\alpha)}$  with probability at least  $1 - \alpha$ . Consequently, satisfying the condition

$$d - 2\sqrt{d \ln(1/\alpha)} \geq (1 - \varepsilon_0)^2 d \Rightarrow d \geq \frac{4 \ln(1/\alpha)}{(2\varepsilon_0 - \varepsilon_0^2)^2}$$

Instantiating this with a high confidence level  $\alpha = 10^{-6}$  and a tight slack  $\varepsilon_0 = 0.05$  requires  $d \gtrsim 5810$ , confirming that the concentration assumption holds robustly for models with parameters on the order of mere thousands.

**Dataset Privacy:**  $\mathcal{A}$  can manipulate the training data to falsify the model ownership, since the  $\mathcal{A}$  may own the labeled dataset. Synthetic data or data reordering results in a non-unique training process for the final ML model of  $\mathcal{P}$ , thereby compromising the integrity of training computation (**M1**). However,  $\mathcal{A}$ 's synthesized weight checkpoint matrix is prone to sudden jumps in the specification of weight changes, causing the verifier to mark the checkpoint as suspicious via  $\text{TS}_{SWCD}$ . Moreover, due to the binding and hiding properties, CP commits all attributes of CTR without revealing any specific data. Without leaking dataset, the specific data point to which the weight checkpoint corresponds cannot be known by the adversary. It ensures that an adversary cannot construct synthetic data points to produce a specific gradient.

**Security of distance from initial weight to final weight:** By placing  $\mathbf{W}[0]$  in a favorable position,  $\mathcal{A}$  can steer the local ML training process towards its desired final result  $f_{W_P}$ , thereby forging a training history to cheat the  $\mathcal{V}$  (**M2**). However,  $\text{TS}_{RIIW}$  ensures each weight checkpoint parameter of the ML model has been randomly initialized from GMM distribution, while it also guarantees the independence between any two weight parameters of the initial model.

**Security of monotonicity of weight checkpoint distance:**  $\mathcal{A}$  can fabricate a sequence of models with varying validation performance and distances from the final model, even without access to the dataset (**M3**). We should note that the trajectory of ML models forged through this attack cannot simultaneously satisfy  $\text{TS}_{RIIW}$  and  $\text{TS}_{MWCD}$ . Due to the randomness involved in  $\text{TS}_{RIIW}$ , the adversary must choose a randomly initialized model and interpolate it with the stolen model.

However, the weight distance between a randomly initialized model and a converged model is larger than the weight distance between the real initial model and the converged model. For sufficiently complex ML models trained with a small learning rate, the optimization starts from the given initialization, is guided by the gradient toward a nearby region of interest, and finally converges to a local minimum that is still close to the starting point. By the central limit theorem, the distance ( $DL(\tilde{\mathbf{W}}[0], \mathbf{W}[P])$ ) between a randomly initialized model (weight matrix  $\tilde{\mathbf{W}}[0]$ ) and the converged model (weight matrix  $\mathbf{W}[P]$ ) follows a normal distribution whose standard deviation is inversely proportional to the square root of the number of parameters. For large ML models, the probability of randomly finding an initial model sufficiently close to the final model is low due to small deviation. Moreover, the distance between the real initial model and the converged model is accumulated step by step along the same training trajectory. By contrast, a model that is independently

and randomly initialized does not lie on this trajectory. Hence, the diminishing weight distance defends interpolation attacks.

## B. Security of Multi-Linear Tree-Baed Matrix Commitment

Analysis on threats (**M4**, **M5**) is as follows.

**Correctness of MTMC:** *Correctness* is guaranteed during the processes of opening, aggregating, and updating a commitment. *Correctness* formally requires as follows:

$$\Pr [\text{MTMC.Verify}_{\text{ppmc}}(\text{MCom}, i, n, P_{i,n}, w_{i,n}, \text{RPK}) = 1] = 1. \quad (13a)$$

$$\Pr [\text{MTMC.Verify}_{\text{ppmc}}(\text{MCom}, i, N, P_{i,N}, w_{i,n}, \text{RPK}) = 1] = 1. \quad (13b)$$

$$\Pr [\text{MTMC.Verify}_{\text{ppmc}}(\text{MCom}', i, N, P'_{i,N}, w'_{i,n}, \text{RPK}) = 1] = 1 \quad (13c)$$

Equations (13 a), (13 b) and (13 c) denote the *Correctness* of individual proof  $P_{i,n}$ , aggregated proof  $P_{i,N}$  and updated proof  $P'_{i,N}$ , respectively. The certificates are stored on the distributed ledger, so adversary cannot fail the verification of certificates from honest clients by attacking the central cloud (**M4**). To ensure successful verification of model weights watermarked with honestly-generated keys (**P1**), all algorithms in the MTMC scheme should be executed correctly.

**Opening Correctness of MTMC:**  $\forall$  security parameter  $\lambda$ , a random position  $(i, n) \in [L] \times [N]$  of a  $L \times N$  matrix  $\mathbf{W}$ ,

$$\exists \Pr \left[ \begin{array}{l} pp \leftarrow \text{MTMC.Gen}(1^\lambda, 1^L, 1^N), \\ \text{MCom} \leftarrow \text{MTMC.Com}_{\text{ppmc}}(\mathbf{W}), \\ P_{i,n} \leftarrow \text{MTMC.Open}_{\text{ppmc}}(w_{i,n}, i, n), \\ 1 \leftarrow \text{MTMC.Verify}_{\text{ppmc}} \left( \begin{array}{l} \text{MCom}, (i, n), \\ w_{i,n}, P_{i,n} \end{array} \right) \end{array} \right] = 1. \quad (14)$$

*Opening Correctness* enables  $\mathcal{V}$  to accurately ascertain the weight values, mitigating the risks of deception or manipulation during the verification process.

**Aggregation Correctness of MTMC:** For any security parameter  $\lambda$ , a random position  $(i, n) \in [L] \times [N]$  of a  $L \times N$  matrix  $\mathbf{W}$ , *Aggregation Correctness* is formally quantified as:

$$\Pr \left[ \begin{array}{l} \text{ppmc} \leftarrow \text{MTMC.Gen}(1^\lambda, 1^L, 1^N), \\ P_{i,n} \leftarrow \text{MTMC.Open}_{\text{ppmc}}(w_{i,n}, i, n), \\ (P_{i,1}, \dots, P_{i,N}) \leftarrow \text{MTMC.OpenRAII}_{\text{ppmc}}(\mathbf{W}, i), \\ P_{i,N} \leftarrow \text{MTMC.AggRow}_{\text{ppmc}} \left( \begin{array}{l} [w_{i,n}]_{n \in N}, i, \\ [P_{i,n}]_{n \in N}, N \end{array} \right), \\ 1 \leftarrow \text{MTMC.Verify}_{\text{ppmc}} \left( \begin{array}{l} \text{MCom}, i, N, \\ P_{i,N}, w_{i,n}, \text{RPK} \end{array} \right) \end{array} \right] = 1. \quad (15)$$

*Aggregation Correctness* holds if the aggregate vector accurately represents all client training computations.

**Commitment Update Correctness of MTMC:** For any security parameter  $\lambda$ , a random position  $(i, n) \in [L] \times [N]$  of a  $L \times N$  matrix  $\mathbf{W}$ , once the matrix  $\mathbf{W}'$  is identical to  $\mathbf{W}$  except  $w'_{i,n} =$



TABLE VII  
COMPARISON OF MSE AND LABEL RECOVERY ACCURACY FOR DIFFERENT  
PRIVACY-PRESERVING ML SCHEMES (BATCH SIZE = 16)

| Model & Dataset     | Method         | Gradient Inversion Attack |                   |
|---------------------|----------------|---------------------------|-------------------|
|                     |                | MSE                       | Recovery Accuracy |
| LeNet & Cifar10     | TSs (baseline) | 0.180                     | 37.5 %            |
|                     | TSs+DP-SGD     | 0.257                     | 25.0 %            |
|                     | TSs+MPC        | 0.258                     | 0 %               |
|                     | MAMMON         | 0.257                     | 0 %               |
| ResNet18 & Cifar100 | TSs (baseline) | 1.561                     | 12.5 %            |
|                     | TSs+DP-SGD     | 1.648                     | 0 %               |
|                     | TSs+MPC        | 1.589                     | 0 %               |
|                     | MAMMON         | 1.590                     | 0 %               |

$w_{i,n} + \xi$ , then we formally quantify the *Commitment Update Correctness* as (16):

$$Pr \left[ \begin{array}{l} pp_{MC} \leftarrow \text{MTMC.Gen}(1^\lambda, 1^L, 1^N), \\ MCom \leftarrow \text{MTMC.Com}_{pp_{MC}}(\mathbf{W}), \\ MCom'' \leftarrow \text{MTMC.ComUpdate}_{pp_{MC}}(MCom, i, n, \xi), \\ MCom' \leftarrow \text{MTMC.Com}_{pp_{MC}}(\mathbf{W}'), \\ MCom'' = MCom' \end{array} \right] = 1. \quad (16)$$

Moreover, the update maintains consistency, ensuring that the modified commitment accurately reflects the intended changes made to the original value.

**Soundness of MTMC:** *Soundness* ensures that the model owner  $\mathcal{V}$  cannot be cheated in the case of an adversary attempting to forge conflicting certificates  $\text{Cert}(f_{W_P}, MCom, SIG_A)$  or invalid certificates (**M5**). Namely, it is required that for each Probabilistic Polynomial Time (PPT) algorithm  $\mathcal{A}$ , the chance of winning MTMC is negligible: Our proposed MTMC scheme is sound (**P2**) in the case where (17) holds, as MTMC can prevent  $\mathcal{A}$  from generating inconsistent certificates for  $w_t \neq w'_t$  at the same position  $t$ .

**Non-stealability of MTMC.** *Non-stealability* means that an adversary  $\mathcal{A}$  who knows the RPK but not the RSK would not be able to claim that this  $\mathcal{A}$  performs the training tasks. Formally, for any security parameter  $\lambda$ , a  $L \times N$  weight matrix  $\mathbf{W}$ , the

chance for each PPT algorithm  $\mathcal{A}$  to win Nonstealability of MTMC is negligible:

$$Pr [\text{Nonstealability}_{MTMC}^{\mathcal{A}}(\lambda, L, N) = 1] \leq \text{negl}(\lambda). \quad (18)$$

**Nonstealability $_{MTMC}^{\mathcal{A}}(\lambda, L, N)$ :**

- 1) Initialize public parameter:  
 $pp_{MC} \leftarrow \text{MTMC.Gen}(1^\lambda, 1^L, 1^N)$ .
- 2) Generate weight vector  $\mathbf{w}_{i \in [L]}$ :  
 $\mathbf{w}_i = [w_{i,1}, w_{i,2}, \dots, w_{i,N}] \in_R \mathbb{Z}_p^N$ .
- 1) Compute local vector commitment:  
 $lvc \leftarrow \text{MTMC.VecCom}_{pp_{MC}}(\mathbf{w}_i)$ .
- 2) Initialize random oracle  $H_{pp_{MC}, \mathbf{w}_i}^{\text{MarkPP}}(\cdot, \cdot)$  with  $M = \emptyset$ .
- 1) Call the adversary algorithm:  
 $(n, \text{RPK}^*, \mathbf{P}^*) \leftarrow \mathcal{A}_{H_{pp_{MC}, \mathbf{w}_i}^{\text{MarkPP}}(\cdot, \cdot)}(pp_{MC}, \mathbf{w}_i)$ .
- 1) Verify the local vector commitment  $lvc$  and return ACCEPT if and only if:  $\text{MTMC.VecComVer}_{pp_{MC}}(lvc, n, (w_{i,n}, \text{RPK}^*), \mathbf{P}^*) = 1 \wedge (\cdot, \text{RPK}^*) \notin M$ .

**$H_{pp_{MC}, \mathbf{w}_i}^{\text{MarkPP}}(o, \text{RPK})$ :**

- 1) If  $\text{RPK} = \perp$  or  $(\text{RSK}, \text{RPK})$  cannot be found in  $M$ :  
 $(\text{RSK}, \text{RPK}) \leftarrow \text{MTMC.KeyGen}_{pp_{MC}}(1^\lambda)$ ;  
 $M \leftarrow M \cup (\text{RSK}, \text{RPK})$ .
- 2) Generate watermark public parameter:  
 $pp_{WMC} \leftarrow \text{MTMC.MarkPPGen}(pp_{MC}, \text{RSK})$ .
- 1) Compute proof  $\mathbf{P} = [lip_{i,1}, lip_{i,2}, \dots, lip_{i,N}]$  for  $lvc_i$ :  
 $\mathbf{P} \leftarrow \text{MTMC.OpenRAI}_{pp_{WMC}}(o, \mathbf{w}_i)$ .
- 1) Return  $(\mathbf{P}, \text{RPK})$ .

These steps collectively define and verify the *Non-stealability* of MTMC by demonstrating that an adversary cannot successfully steal the commitment within PPT time.

In order to prevent adversaries from removing cryptographic signatures, our scheme incorporates a watermark directly into the commitment, ensuring the non-stealability of the data. In MTMC, the operation for  $\mathcal{A}$  to steal the commitment is to remove  $\theta$  from the watermarked proof  $\pi_i^\theta$ . However, this attack requires

$$Pr \left[ \begin{array}{l} pp_{MC} \leftarrow \text{MTMC.Gen}(1^\lambda, 1^L, 1^N), \\ \left( \begin{array}{l} MCom, \\ \left( \begin{array}{l} (i, N_1, w_{i,n}), \\ (\text{RPK}_n, P_{i,N_1}) \end{array} \right)_{n \in N_1}, \\ \left( \begin{array}{l} (i, N_2, w'_{i,n'}), \\ (\text{RPK}_{n'}, P'_{i,N_2}) \end{array} \right)_{n' \in N_2} \end{array} \right) \leftarrow \mathcal{A}(1^\lambda, pp_{MC}) : \\ 1 \leftarrow \text{MTMC.Verify}_{pp_{MC}} \left( MCom, N_1, \text{RPK}_n, \begin{array}{l} P_{i,N_1}, w_{i,n}, i \end{array} \right)_{n \in N_1} \wedge \\ 1 \leftarrow \text{MTMC.Verify}_{pp_{MC}} \left( MCom, N_2, \text{RPK}_{n'}, \begin{array}{l} P'_{i,N_2}, w'_{i,n'}, i \end{array} \right)_{n' \in N_2} \wedge \\ \exists j \in N_1 \cap N_2, w_{i,j} \neq w'_{i,j'} \end{array} \right] \leq \text{negl}(\lambda). \quad (17)$$

the exponent value  $\theta^{-1}$ , which  $\mathcal{A}$  does not know. Specifically, the private key RSK ( $\theta$ ) is embedded into the commitment as an identity-bound watermark. Since this watermark is part of the commitment,  $\mathcal{A}$  that knows only the RPK cannot remove it, cannot replace it with its own identifier, and cannot generate a new valid certificate for the same weights. The only way to produce such a commitment is to recover the RSK ( $\theta$ ), which is computationally infeasible under the security parameter  $\lambda$ . Therefore,  $\mathcal{A}$  cannot steal the certificate by removing or replacing the  $\mathcal{P}$ 's signature (M4).

**Homomorphism of MTMC:** We consider the multi-linear trees of two vectors,  $\mathbf{ve}_1$  and  $\mathbf{ve}_2$ . The multi-linear tree of  $\mathbf{ve}_1 + \mathbf{ve}_2$  contains specific polynomial commitments  $\tilde{w}_{\rho_{i,n}}$  at the  $i$ -th leaf node of the  $n$ -th path. These commitments are computed as the product of the commitments at the corresponding  $(i, n)$ -th nodes in the multi-linear trees of  $\mathbf{ve}_1$  and  $\mathbf{ve}_2$ . For any  $i$ -th node ( $\rho_{i,n}$ ) of  $n$ -th path in the tree, such commitment computation can be quantified as:

$$lvc(\tilde{w}_{\rho_{i,n}}'') = lvc(\tilde{w}_{\rho_{i,n}} + \tilde{w}_{\rho_{i,n}}') = lvc(\tilde{w}_{\rho_{i,n}}) \cdot lvc(\tilde{w}_{\rho_{i,n}}') \quad (19)$$

where  $\tilde{w}_{\rho_{i,n}}$ ,  $\tilde{w}_{\rho_{i,n}}'$ ,  $\tilde{w}_{\rho_{i,n}}''$  denote the polynomial commitments stored on  $\rho_{i,n}$  in the multi-linear tree for  $\mathbf{ve}_1$ ,  $\mathbf{ve}_2$  and  $\mathbf{ve}_1 + \mathbf{ve}_2$ , respectively. Due to homomorphism, our scheme supports secure aggregation of local ML models from multiple clients.

## VI. EXPERIMENTS AND EVALUATION

### A. Theoretical Analysis

**Time Complexity:** Table I compares the computational costs of DML ownership verification schemes along three operations: Proof Generation, Proof Update, and Proof Verification. Let  $M = L \times N$  denote the size of the model weight matrix  $\mathbf{W}$ , and let  $S$  denote the number of training steps for the underlying ML computation. Merkle SNARK constructions incur linear complexity in the model size for all three operations, i.e.,  $O(M)$ , because every update must modify the corresponding Merkle path. Hyperproofs [30] improve update and verification to logarithmic time,  $O(\log M)$ , by precomputing a PST-style tree of commitments; however, the proving side still needs to materialize all  $M$  leaves, which leads to a superlinear term  $(M) \cdot O(M)$  during proof generation. The zkPoT [26] treats the ML training procedure itself as the arithmetic/boolean computation to be proved in zero knowledge; as a result, both proof generation and proof update scale with the training trace size, namely  $O(S \cdot M \cdot \log(S \cdot M))$ , while proof verification only needs to check a batched opening and therefore remains linear in the trace,  $O(S \cdot M)$ . PoLO [33] keeps hash-based watermarking operations linear in the number of parameters ( $O(M)$ ) and its verification cost splits into the forward-pass cost  $O(C_{fwd})$  plus an additional linear term to validate the chained watermark. Finally, MAMMON inherits the linear commitment cost  $O(M)$  and supports proof updates in only  $O(L + \log N)$  time, while individual proof verification is  $O(L \cdot \log N)$  because it need to verify the path from leaf to root.

**Key Size:** In MAMMON, the size of the public proving material is determined by the two types of proofs. First, for *lvc* and *lip*, the multi-linear tree has to maintain all  $2N - 1$  nodes; consequently, generating an *lvc* or an *lip* touches  $2N - 1$  elements in  $\mathbb{G}_1$ . Second, for *gip* the prover needs an  $L \times L$  Toeplitz matrix for each column and has to traverse all  $N$  columns. Since an  $L \times L$  Toeplitz matrix can be represented using  $2L - 1$  group elements, generating a *gip* involves  $N \cdot (2L - 1)$  elements in  $\mathbb{G}_1$ . Combining both parts, the proving key size is  $(2^N N - 1) + N \cdot (2^L L - 1) = (2^N N \cdot L + N - 1)$  elements in  $\mathbb{G}_1$ . The update key has the same structure and therefore has the same size, i.e.,  $(2^N N \cdot L + N - 1)$  elements in  $\mathbb{G}_1$ . For the verification, (9a) requires verifier to hold  $(\log N)$  elements in  $\mathbb{G}_2$ , while (9b) requires an additional  $L + 2$  elements in  $\mathbb{G}_2$ . Hence, the overall verifying key size is  $(\log N + L + 2)$  elements in  $\mathbb{G}_2$ .

**Proof Size:** The matrix commitment (MCom) has size 1 element in  $\mathbb{G}_1$ . An individual proof for a single matrix consists of three components: *lvc*, *gip*, and *lip*. Specifically, *lvc* has size 1 element in  $\mathbb{G}_1$ , *gip* also has size 1 element in  $\mathbb{G}_1$ , while *lip* corresponds to one path in the multi-linear tree and therefore has size  $\log N$  elements in  $\mathbb{G}_1$ . Hence, an unaggregated individual proof ( $P_{i,n}$ ) has size  $1 + 1 + \log N = (\log N + 2)$  elements in  $\mathbb{G}_1$ . Moreover, when  $L$  such *lip* proofs are aggregated, the resulting proof  $P_{i,N}$  has size  $\log(L \log N)$  group elements.

### B. Experiments for Multi-Linear-Based Matrix Commitment

**1) Experiment Configuration:** We implement the matrix commitment scheme of MAMMON and apply the multi-linear tree architecture. MAMMON is implemented in Golang language with the following stack: we utilize mcl cryptography library<sup>1</sup> as the underlying algebra backend for finite fields  $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ , BLS12-381 as a pairing-friendly elliptic curves, and polynomial evaluation operations. Moreover, we realize an IPA-based local index proof aggregation refers to Hyperproofs code.<sup>2</sup> The MTMC scheme running environment of our experiment is on the Intel(R) Xeon(R) Gold 6150 CPU@2.70 GHz with 18 cores and 256 GB memory. We run 10 times for each experiment and record their average.

On one hand, we set a series of weight matrix sizes, denoted by  $L = N \in \{2^n | n \in \mathbb{Z}, 2 \leq n \leq 9\}$  and the corresponding height of the multi-linear tree as  $h \in \{n | n \in \mathbb{Z}, 2 \leq n \leq 9\}$ <sup>3</sup>. Thus, we evaluate time costs of all algorithms in MTMC scheme for various matrix sizes (see Algorithm 2). On the other hand, to simulate the process of local ML weight training, we evaluate the performance of multi-linear tree-based local vector commitment scheme for eight typical ML network models, i.e., LeNet, ResNet18, AlexNet, VGG16, CLIP, Bert-Large, Stable Diffusion v1.4, and Whisper-Large. Table II shows the model architectures and total weight parameter counts ( $N$ ) of evaluated ML models.

We measure the time costs and storage costs for weight updates and aggregation to evaluate the *Maintainability* and *Aggregability* of MTMC. We vary the number of updated/aggregated

<sup>1</sup>The library is available at <https://github.com/herumi/mcl.git>

<sup>2</sup>The code is available at <https://github.com/hyperproofs/hyperproofs-go.git>

<sup>3</sup>The weight matrix involves  $L \times N$  weight parameters and  $N = 2^h$ .

weights  $\eta \in \{2^n | n \in \mathbb{Z}, 2 \leq n \leq 11\}$  and set tree heights  $h \in \{16\} \cup \{n | n \in \mathbb{Z}, 25 \leq n \leq 31\}$  to correspond with LeNet, ResNet18, AlexNet, VGG16, CLIP, Bert-Large, Stable Diffusion v1.4, and Whisper-Large, respectively. We compare MTMC against a *Merkle SNARK* baseline implemented using the *bellman-bignat* library.<sup>4</sup> Since the baseline requires over 6 hours for initialization when the vector length  $N \geq 128$ , we restrict the comparative evaluation to a smaller scale. Specifically, we fix  $\eta = 16$  and vary  $N \in \{8, 16, 32, 64\}$ .

2) *Results*: MAMMON is operated by a single-threaded manner and costs are evaluated within four main phases of MTMC, including Proof Generation, Proof Update, Proof Aggregation, and Proof Verification. Table III presents results of time costs across varying tree heights ( $h$ ). While proof generation scales with matrix size, it remains a one-time initialization cost; conversely, the aggregation overhead exhibits sub-linear growth relative to the tree height  $h$ , ensuring efficient *Aggregatability* for matrix commitments. Notably, the commitment update operation (MTMC.ComUpdate) remains in the millisecond regime across all evaluated sizes, ensuring exceptional *Maintainability* for high-frequency weight adjustments. Furthermore, even at a large size of  $L = N = 512$ , aggregated proof verification completes within roughly 1.6 s, supporting MTMC's feasibility for large deployments.

*Evaluation Results for Local Model Training*: Fig. 5 presents the computational overhead of the MAMMON scheme across eight ML models, including aggregation, update, and verification time costs. The IPA-based aggregation overhead follows  $O(\eta \cdot \log N)$  complexity. For example, in LeNet, the aggregation cost rises from 0.087 s to 38.444 s as  $\eta$  increases from  $2^2$  to  $2^{11}$ . Meanwhile, the update stage remains efficient even for very large architectures like Whisper-Large ( $N = 2^{31}$ ), with latency capped at 7.71 s. Overall, these results indicate that MAMMON concentrates the dominant computation in commitment generation operations while keeping verification overhead low, thereby supporting scalable weight-iteration workloads.

Table IV summarizes the storage costs of commitments, proofs, and key materials for multilinear tree heights  $h \in \{26, 27, 28, 29, 30\}$ . Specifically, *MCom*, *lvc*, and *gip* are consistently about 0.047 KiB (48 bytes), which matches the serialized size of a single  $G_1$  group element in our implementation, and thus remain unchanged across different  $h$ . The only height-dependent proof term is *lip*, which increases from 1.219 KiB (at  $h = 26$ ) to 1.406 KiB (at  $h = 30$ ), reflecting the longer height-indexed information carried in the *lip* proof. For key sizes of *lvc*, the *univ. key mat.* remains constant at 216 MiB for all  $h$  because our IPA-based local vector commitment treats the expensive setup as a one-time cost in practice. Specifically, universal key material is generated by a sufficiently large upper-bound parameter and then is reused across instances of different vector sizes<sup>5</sup>. For any smaller vector length  $N$ , the proving key is obtained by simply taking the first  $2N - 1$  elements from

the universal material, avoiding repeated setup when evaluating different weight vectors. Moreover, the universal material consists of a commitment key (72 MiB) and KZG key material (144 MiB), while the verification key grows mildly with height from 7.312 KiB at  $h = 26$  to 8.437 KiB at  $h = 30$ . Overall, these results indicate that the commitment and proof storage remains limited, and the key material for vector commitment can be amortized via a universal setup, making the scheme practical for large-scale training scenarios that require committing to and verifying training histories.

*Comparison with Merkle SNARK*. We evaluate MTMC against a Rust-based *Merkle SNARK* implementation (Bellman-Bignat) using Sha256, Pedersen, and Poseidon hash functions and summarize the results in Table V. Compared to *Merkle SNARK* implementations, MTMC demonstrates superior efficiency in parameter and proof generation, outperforming even the most optimized hash functions like Poseidon across most matrix scales. The primary bottleneck for *Merkle SNARK* lies in FFT operations tied to R1CS constraints: while Poseidon is efficient, it lacks the homomorphism required for DML aggregation, and homomorphic alternatives like Pedersen incur prohibitive delays due to massive constraint overhead (2,753 per call). Although *Merkle SNARK* schemes maintain a constant-time verification advantage, this is rendered negligible by the massive latency incurred during their generation phase. Overall, MTMC provides a far more practical solution for high-frequency dynamic updates in DML, whereas *Merkle SNARK* remains impractical due to its prohibitive computational requirements.

### C. Experiments for Local ML Models

1) *Dataset and Models*: Our experiments involve two computer vision datasets, CIFAR10 and CIFAR100, and a segmentation/detection dataset, PASCAL VOC2012. For classification tasks, we train LeNet, ResNet18, AlexNet, and VGG16 on the CIFAR10 and CIFAR100 datasets. For semantic segmentation, we implement a UNet model for MAMMON on the VOC2012 dataset, and for object detection, we employ a YOLOv8n model on the same dataset. In addition, to evaluate the impact of MAMMON on training accuracy, we have conducted experiments using ResNet20, ResNet32, and ResNet44 on CIFAR10, and ResNet18, ResNet34, and ResNet50 on CIFAR100. All experiments are carried out on a system equipped with an Intel(R) Xeon(R) Platinum 8352 V CPU @ 2.10 GHz and an RTX 5090 GPU. It should be emphasized that our approach does not modify the model architecture or parameters; it only records intermediate variables during the training process.

2) *Adversarial Attack Settings*: Assuming  $\mathcal{A}$  has white-box access to the structure and weights of  $f_{WP}$ , and may also possess an auxiliary non-labeled dataset. To evaluate the robustness of MAMMON against forgery attempts, we implement three distinct attack strategies as follows:

*Checkpoint Forgery Attack (CFA)*:  $\mathcal{A}$  fabricates a sequence of pseudo checkpoints by linearly combining the weights of a randomly initialized model and the final converged model ( $f_{WP}$ ). Specifically, the parameters at step  $t$  are constructed

<sup>4</sup><https://github.com/alex-ozdemir/bellman-bignat>

<sup>5</sup>In general, the upper-bound parameter should cover  $h \times \eta$ . We instantiate it as  $2^{19}$  in our implementation.



as

$$\widetilde{\mathbf{W}}[t] = (1 - \mu) \widetilde{\mathbf{W}}[0] + \mu \mathbf{W}[P], \quad t \in [0, P],$$

where  $\widetilde{\mathbf{W}}[0]$  denotes the randomly initialized weights of the  $\mathcal{A}$ ,  $\mathbf{W}[P]$  represents the converged weights of the victim model  $f_{\mathbf{W}_P}$ , and  $\mu \in [0, 1]$  is the interpolation coefficient [27]. By sweeping  $\mu$ ,  $\mathcal{A}$  obtains a series of checkpoints without requiring access to the original dataset.

**Regulated Extraction Attack (REA):** RCA is similar to zero-shot knowledge distillation attacks [34]. Since  $\mathcal{A}$  does not have access to the labels of the original dataset, it defines the knowledge distillation loss as the KL divergence between the teacher model and the student model.

**Gradient Inversion Attack (GIA):** GIA [35] assumes the  $\mathcal{A}$  observes the gradients of one training step and then treats the input data and labels as optimization variables, iteratively updating these dummy inputs so that their gradients match the observed ones and thus approximately recover the original training samples.

**3) Experiment Results: Experiment Results of CFA and REA:** To evaluate the robustness of the three TSs, we conduct experiments on four image classification tasks on CIFAR10 and CIFAR100 (LeNet, ResNet18, and AlexNet), a semantic segmentation task on PASCAL VOC2012 (UNet), and an object detection task (YOLOv8n). For each task, we compare the TSs of clean MAMMON checkpoints with those of forged checkpoints generated by CFA and REA.

Concretely,  $\text{TS}_{\text{SWCD}}$  is defined as the maximum layer-wise parameter distribution distance between any pair of consecutive checkpoints.  $\text{TS}_{\text{RIIW}}$  is decomposed into two sub-metrics: (1)  $\text{TS}_{\text{RIIW}}(\text{DP})$ , measured as the distance between the empirical distribution of the initial parameters and the GMM distribution; and (2)  $\text{TS}_{\text{RIIW}}(\text{COV})$ , quantified as the maximum explained variance ratio obtained by PCA within each parameter group of the initial model.  $\text{TS}_{\text{MWCD}}$  is computed as the weight distance between the initial and final checkpoints. In addition, we assess the monotonicity of validation accuracy, defined as the Spearman rank correlation between validation accuracy and the training epoch index. Under standard training, the optimization objective typically encourages the training loss to decrease monotonically until convergence. When the validation set shares a similar distribution with the training set and the model does not severely overfit, the validation accuracy is expected to be non-decreasing on average as training progresses. Although complex tasks may exhibit local non-monotonic behavior due to optimization instability and local minima, the overall trend of validation accuracy is usually close to monotonic. Therefore, a higher accuracy-monotonicity value indicates a greater likelihood that the model has undergone correct training.

Table VI shows the values of  $\text{TS}_{\text{SWCD}}$ ,  $\text{TS}_{\text{RIIW}}$ ,  $\text{TS}_{\text{MWCD}}$ , and validation accuracy under attacks with interpolation coefficient  $\mu = 0.05$ . We treat  $\text{TS}_{\text{RIIW}}$  as satisfied only when both the  $\text{TS}_{\text{RIIW}}(\text{DP})$  and  $\text{TS}_{\text{RIIW}}(\text{COV})$  components meet their respective thresholds. Furthermore, to make  $\text{TS}_{\text{MWCD}}$  statistically meaningful, we construct a set of randomly initialized models with the same architecture and compute the mean (DisMean) and standard deviation (DisStd) of their  $\text{TS}_{\text{MWCD}}$

values. We accept  $\text{TS}_{\text{MWCD}}$  only if  $\text{TS}_{\text{MWCD}} < \text{DisMean} - 5 \times \text{DisStd}$ . Under CFA, the  $\mathcal{A}$  interpolates between a randomly initialized model and a stolen final model. In order to keep  $\text{TS}_{\text{SWCD}}$  and  $\text{TS}_{\text{RIIW}}$  within the acceptable threshold, the interpolation coefficient must be small (e.g.,  $\mu = 0.005$ ). However, as shown in Table VI, in this settings the distance between the randomly initialized model and the final model is substantially larger than that between the correct initial and final checkpoints, causing  $\text{TS}_{\text{MWCD}}$  to fail. Although some checkpoints forged by CFA satisfy  $\text{TS}_{\text{SWCD}}$ , such as those for CIFAR100 with ResNet18/LeNet and for VOC2012 with YOLOv8n, their  $\text{TS}_{\text{RIIW}}$  values differ substantially from those of correctly trained MAMMON models. For CIFAR100 with ResNet18 and LeNet, the  $\text{TS}_{\text{RIIW}}(\text{DP})$  is much larger, and for VOC2012 with YOLOv8n, the  $\text{TS}_{\text{RIIW}}(\text{COV})$  is considerably higher. Consequently,  $\text{TS}_{\text{RIIW}}$  is not satisfied, and the three TSs still reliably flag these checkpoints as forged.

In an REA, the  $\mathcal{A}$  trains a student model with the same architecture as the protected model, initialized to match the GMM distribution, and then attempts to forge a series checkpoints that satisfies  $\text{TS}_{\text{SWCD}}$  and  $\text{TS}_{\text{RIIW}}$ . According to Table VI, every REA-forged checkpoints that passes  $\text{TS}_{\text{RIIW}}$  is rejected by  $\text{TS}_{\text{MWCD}}$ . Conversely, for the UNet and YOLOv8n settings where  $\text{TS}_{\text{MWCD}}$  happens to be small enough, the corresponding  $\text{TS}_{\text{RIIW}}(\text{COV})$  is still noticeably larger than that of genuinely trained models, leading to a failure of  $\text{TS}_{\text{RIIW}}$ . Moreover, even when REA produces checkpoints whose  $\text{TS}_{\text{MWCD}}$  values resemble those of real training, their accuracy-monotonicity is consistently worse than that of the clean models. This degradation arises because the student model is trained mainly to mimic the teacher's logits rather than to fit ground-truth labels, and the limited or synthetic supervision disrupts the expected monotonic growth pattern of validation accuracy. Consequently, these results demonstrate that the proposed TSs remain effective at detecting forged checkpoints under both CFA and REA.

Moreover, we evaluate how launching CFA with different interpolation coefficients ( $\mu$ ) affects the distance between an initialized model and the converged model, to further demonstrate the effectiveness of  $\text{TS}_{\text{MWCD}}$ . Fig. 6 shows the values of  $\text{TS}_{\text{MWCD}}$  for different datasets and models after launching CFA multiple times. For each interpolation coefficient  $\mu$ , we conduct 5 independent CFA runs. In addition, we note that all forged weight checkpoints involved in Fig. 6 successfully pass the  $\text{TS}_{\text{RIIW}}$  verification. As shown in Fig. 6, in order to satisfy the randomness and independence requirements on weight initialization imposed by  $\text{TS}_{\text{RIIW}}$ , the distance ( $DL(\mathbf{W}[0]', \mathbf{W}[P])$ ) of forged checkpoints after CFA is significantly larger than the corresponding distance ( $DL(\mathbf{W}[0], \mathbf{W}[P])$ ) in clean training histories. Concretely, under a clean ML training procedure with a small learning rate, weight updates progress incrementally along one coherent optimization path, leading the model to a local minimum that is still near its original initialization. By comparison, randomly initialized models that comply with  $\text{TS}_{\text{RIIW}}$  are generated independently and do not reside on this optimization path.

**Experiment Results of GIA:** Since MAMMON is an ownership protection scheme for local model training, we implement

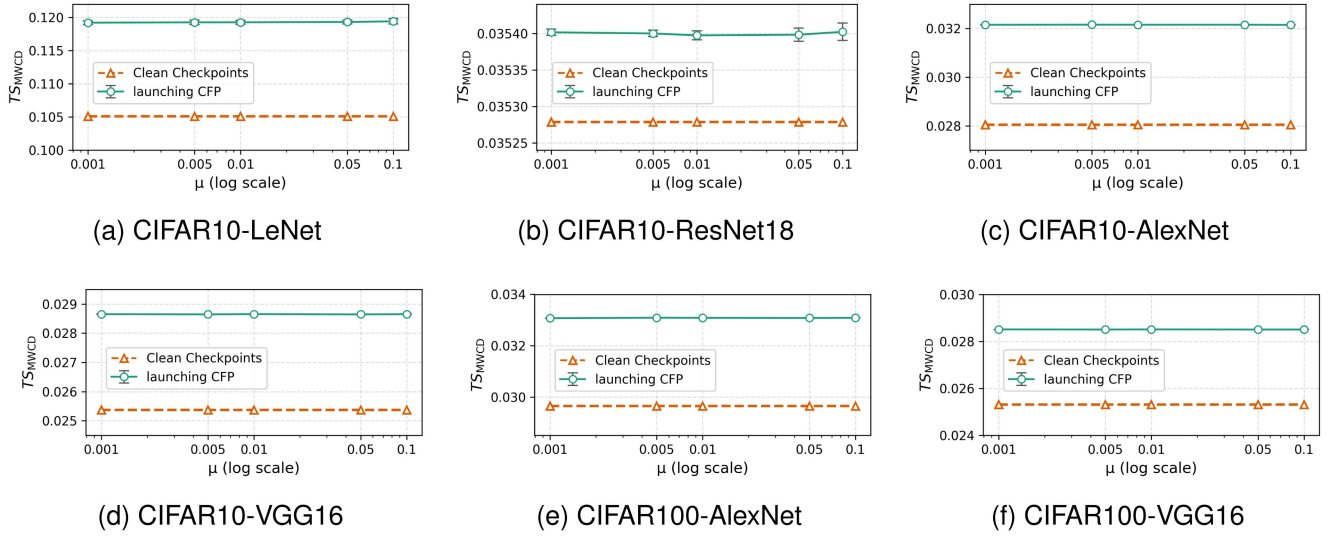


Fig. 6. The comparison of the distance between the initial model and the converged model ( $TS_{MWCD}$ ) after launching CFA attacks with different values of  $\mu$ . “Clean” denotes the original and unattacked weight checkpoint.

TABLE VIII  
THE ACCURACY OF LOCAL ML TRAINING OF MAMMON

| Dataset  | Models    | Schemes  |        |        | Ours vs Original | Ours vs PoL |
|----------|-----------|----------|--------|--------|------------------|-------------|
|          |           | Original | PoL    | Ours   |                  |             |
| CIFAR10  | Resnet-20 | 91.73%   | 89.54% | 91.77% | +0.04%           | +2.23%      |
|          | Resnet-32 | 92.97%   | 90.02% | 92.67% | -0.30%           | +2.65%      |
|          | Resnet-44 | 94.16%   | 90.46% | 92.56% | -1.60%           | +2.10%      |
| CIFAR100 | Resnet-18 | 75.97%   | 74.47% | 74.53% | -1.44%           | +0.06%      |
|          | Resnet-34 | 76.85%   | 74.87% | 74.93% | -1.92%           | +0.06%      |
|          | Resnet-50 | 78.55%   | 76.94% | 76.96% | -1.59%           | +0.02%      |

the GIA in the local training setting and compare the privacy protection capability of MAMMON with DPSGD [36] and secure Multi-Party Computation (MPC) [37]. We evaluate the privacy protection capability of our matrix commitment scheme using two metrics, Mean Squared Error (MSE) and label recovery accuracy, and we take the LeNet and ResNet18 that implement only the three TSs as the baseline. We note that in both MAMMON and MPC, gradients and model parameters are encrypted, so in principle an  $\mathcal{A}$  cannot extract any useful information (such as tensors for inversion) from the ciphertexts. Therefore, in our experiments we simulate what an  $\mathcal{A}$  can observe under MAMMON and MPC by providing random tensors instead of real gradients. Because an  $\mathcal{A}$  can, in theory, apply forward or backward inference on these random tensors, their security is actually weaker than that of MAMMON and MPC, which never expose such plaintext tensors. Hence, the attack results using random tensors give a conservative lower bound: the real protocols can only offer stronger privacy protection.

Table VII compares the reconstruction Mean Squared Error (MSE) and label recovery accuracy of gradient inversion attacks under different privacy-preserving schemes. For LeNet on CIFAR-10, the TSs baseline attains a low MSE of 0.180 and a high label recovery accuracy of 37.5%, indicating severe privacy leakage. After incorporating MPC or MAMMON, the label recovery accuracy drops to 0% while the MSE increases

compared to the baseline, showing that both methods effectively suppress label leakage. For ResNet18 on CIFAR-100, the TSs baseline still allows non-trivial label recovery (12.5%), whereas all three enhanced schemes (TSs+DP-SGD, TSs+MPC, and MAMMON) reduce the label recovery accuracy to 0%. Overall, across both settings, MAMMON attains MSE values comparable to other privacy mechanisms but completely blocks label recovery, demonstrating its effectiveness against gradient inversion attacks.

*Experiment Results of Local Training:* We conducted comparative experiments across various architectures among MAMMON, PoL [14], and original DNN models, with the results summarized in Table VIII. Since MAMMON preserves the native model structure, its impact on performance is negligible. The results demonstrate that MAMMON maintains accuracy consistent with original models while slightly outperforming PoL, achieving high-fidelity security reinforcement with minimal utility loss. In our experiments, the slightly lower performance of PoL mainly comes from its requirement of high-frequency checkpointing. During one local epoch, PoL must save the model state every  $k$  steps so that the training history can later be reconstructed or verified. When  $k$  is small, the frequent I/O and serialization introduce non-negligible overhead and interrupt the optimizer, which reduces the number of effective updates within the same time budget and makes the gradient and learning-rate schedule less smooth. By contrast, MAMMON only commits checkpoints once per local epoch and only for models that pass the training specifications, so it preserves the accuracy of correct training.

## VII. RELATED WORK

*Ownership Verification for Machine Learning:* Model watermarking is one of exploration directions for verifying ML ownership [38], [39], [40]. For example, [40] tried watermark triggers generated from original training samples to prevent

retraining forgetting, but modifying ML model is generally required, e.g., model outputs, activation functions, or network structures. To avoid modification, similarity checking between the suspect model and the victim model is used for ownership verification [28]. Another method was training dataset that distinguished stolen models by exposing the model owner's private knowledge of the dataset and decision boundaries [41], [42], [43]. However, existing methods encountered limitations caused by the potential exposure of valuable and sensitive data checkpoints. Moreover, embedding watermarks in the global DML model was explored for ownership verification [10]. [44] injected client-distinguishable watermarks without degrading task performance for multi-modal federated learning. [11], [12] explored multi-client watermarking methods for DML ownership verification, while these methods rely on model-side embedding, thus remaining vulnerable to exposure, interference, or attenuation of the private watermark during re-training and aggregation. Differently, our MAMMON was homomorphic and enables the model owner to aggregate commitments from all participating clients with only sub-linear aggregation overhead. MAMMON avoided disclosing clients' private information and remained robust across multiple training rounds without attenuation.

*Verifiable Computation in Cryptography:* Verifiable computation allowed the model owner to outsource computing tasks to an untrustworthy executor by examining the proof of the result [45]. For example, [46] tried to maintain function privacy through oblivious evaluation and batch result verification. [47] was a verifiable aggregation protocol with dimension-independent communication cost and bounded computational overhead in federated learning. Some studies explored privacy-preserving DML training. For instance, [48] turned neural network inference into a two-non-colluding-edge secure protocol by rewriting the model as BNN and using lightweight secret-sharing-friendly operators. [26] improved MPC-in-the-head to realize zero-knowledge proof for model training. However, existing methods failed in embedding the original training algorithm losslessly, as they approximated non-arithmetic operations for arithmetization, which alters training and typically proves only one epoch. By contrast, MAMMON committed to the training history without changing the DML model or its training algorithms.

### VIII. POTENTIAL LIMITATIONS AND FUTURE WORK

While MAMMON demonstrates robust ownership verification on public benchmark datasets, we acknowledge an important limitation: the current TS are mainly designed around the training dynamics of standard public datasets, where optimization trajectories are relatively smooth and well-behaved. Deploying the framework in real-world privacy-sensitive DML domains (e.g., medical or financial applications) introduces additional dynamic challenges. Future work will focus on four directions: data heterogeneity, noisy environments, resource constraints, and stronger privacy protection.

*Data heterogeneity (Non-IID):* Under high heterogeneity, client drift can cause local updates to deviate from the global optimization direction for extended periods, which weakens

the strict monotonicity assumption used in  $TS_{MWCD}$ . Inspired by trajectory matching techniques [49], the future work is to shift from purely geometric/statistical verification to a data-consistency-based criterion. Concretely, we will generate synthetic data from the claimed trajectory and test whether it aligns with the client's declared local heterogeneous distribution. This design reduces reliance on global convergence and improves robustness to statistical heterogeneity.

*Noisy environments with privacy mechanisms:* Privacy-preserving training methods such as DP-SGD introduce random noise into gradients and updates, which can violate the fixed distance thresholds in  $TS_{SWCD}$ . We will therefore replace rigid Euclidean thresholds with information-theoretic metrics that quantify dependence between the data and the model update. In particular, our future work is to use mutual-information-based measures, such as entropy reduction rates, to capture the causal dependence between training data and parameter changes. This shift aims to distinguish honest-but-noisy training from updates dominated by random noise, thereby reconciling privacy noise with integrity verification.

*Resource constraints:* In privacy-sensitive DML, it is often impractical to store or transmit checkpoints at every epoch or round. To improve scalability, we will investigate verification using sampled fragments of the training history (e.g., one checkpoint every  $K$  rounds) and perform trajectory matching via synthetic data [50]. A key goal is to characterize the minimal fragment length required for reliable discrimination. Future work targets conditions where the mutual-information gap between honest training histories and privacy-preserving baselines is statistically significant, while minimizing communication and storage overhead.

*Privacy enhancement via zero-knowledge proofs:* We further plan to encode TS as predicates within zk-SNARKs, enabling verifiable proofs while hiding both the predicate evaluation trace and sensitive data. Compared with fully encoding ML arithmetic operations into zk-SNARK circuits, encoding only the TS is substantially lighter and more efficient. This direction offers a practical trade-off between client privacy protection and ownership verification efficiency.

### IX. CONCLUSION

In this paper, we propose a novel MAMMON scheme to address ownership disputes and ensure computational integrity in DML. It uses a matrix commitment with limited overhead and concise proofs for frequent parameter updates in DML. A maintainable multi-linear tree enables fast proof updates and aggregation. Moreover, MAMMON embeds client identity keys into the matrix commitment and prevents forgery or duplication of commitments. Experiments with Merkle-SNARK hash commitments show that MAMMON preserves computational integrity while improving proof generation and update efficiency.

### REFERENCES

- [1] Y. Ge et al., "OpenAGI: When LLM meets domain experts," in *Proc. 37th Int. Conf. Neural Inf. Process. Syst.*, 2024, vol. 36, pp. 5539–5568.
- [2] X. Ma, G. Fang, and X. Wang, "LLM-Pruner: On the structural pruning of large language models," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2023, vol. 36, pp. 21702–21720.



- [3] R. Zhou, X. Zhang, J. C. S. Lui, and Z. Li, "Dynamic pricing and placing for distributed machine learning jobs: An online learning approach," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 4, pp. 1135–1150, Apr. 2023.
- [4] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.
- [5] T. Eltaras, F. Sabry, W. Labda, K. K. Alzoubi, and Q. Malluhi, "Efficient verifiable protocol for privacy-preserving aggregation in federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 2977–2990, 2023.
- [6] C. Hahn, H. Kim, M. Kim, and J. Hur, "VerSA: Verifiable secure aggregation for cross-device federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 36–52, Jan./Feb. 2023.
- [7] A. Roy Chowdhury, C. Guo, S. Jha, and L. van der Maaten, "ElFFeL: Ensuring integrity for federated learning," in *Proc. ACM SIGSAC Conf. Computer Commun. Secur.*, 2022, pp. 2535–2549.
- [8] J. Tan, N. Zhong, Z. Qian, X. Zhang, and S. Li, "Deep neural network watermarking against model extraction attack," in *Proc. 31st ACM Int. Conf. Multimedia*, 2023, pp. 1588–1597.
- [9] J. Chen, M. Li, Y. Cheng, and H. Zheng, "FedRight: An effective model copyright protection for federated learning," *Comput. Secur.*, vol. 135, 2023, Art. no. 103504.
- [10] J. Liu et al., "TraCemop: Towards federated learning with traceable contribution evaluation and model ownership protection," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 10230–10246, Oct. 2025.
- [11] B. Li, L. Fan, H. Gu, J. Li, and Q. Yang, "FedIPR: Ownership verification for federated deep neural network models," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 4, pp. 4521–4536, Apr. 2023.
- [12] S. Shao et al., "FedTracker: Furnishing ownership verification and traceability for federated learning model," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 1, pp. 114–131, Jan./Feb. 2025.
- [13] Y. Yang, Q. Li, Y. Hong, and B. Wang, "FedGMark: Certifiably robust watermarking for federated graph learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2024, vol. 37, pp. 48971–48995.
- [14] H. Jia et al., "Proof-of-learning: Definitions and practice," in *Proc. IEEE Symp. Secur. Privacy*, 2021, pp. 1039–1056.
- [15] D. Choi, Y. Shavit, and D. Duvenaud, "Tools for verifying neural models' training data," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, vol. 36, pp. 1154–1188.
- [16] K. Abbaszadeh, C. Pappas, D. Papadopoulos, and J. Katz, "Zero-knowledge proofs of training for deep neural networks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2024, pp. 4316–4330.
- [17] G. Tan et al., "Founding zero-knowledge proofs of training on optimum vicinity," in *Proc. ACM SIGSAC Conf. Computer Commun. Secur.*, 2025, pp. 1173–1187.
- [18] A. Thudi, H. Jia, I. Shumailov, and N. Papernot, "On the necessity of auditable algorithmic definitions for machine unlearning," in *Proc. 31st USENIX Secur.*, 2022, pp. 4007–4022.
- [19] C. Fang et al., "Proof-of-learning is currently more broken than you think," in *Proc. IEEE 8th Eur. Symp. Secur. Privacy*, 2023, pp. 797–816.
- [20] R. Zhang, J. Liu, Y. Ding, Z. Wang, Q. Wu, and K. Ren, "Adversarial examples for proof-of-learning," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 1408–1422.
- [21] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza, "Succinct non-interactive zero knowledge for a von Neumann architecture," in *Proc. 23rd USENIX Secur. Symp.*, 2014, pp. 781–796.
- [22] M. R. Albrecht, V. Cini, R. W. F. Lai, G. Malavolta, and S. A. Thyagarajan, "Lattice-based SNARKs: Publicly verifiable, preprocessing, and recursively composable," in *Proc. Annu. Int. Cryptol. Conf.*, 2022, pp. 102–132.
- [23] H. Sun, T. Bai, J. Li, and H. Zhang, "ZKDL: Efficient zero-knowledge proofs of deep learning training," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 914–927, 2024.
- [24] X. Tang, M. Shen, Q. Li, L. Zhu, T. Xue, and Q. Qu, "PILE: Robust privacy-preserving federated learning via verifiable perturbations," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 6, pp. 5005–5023, Nov./Dec. 2023.
- [25] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "VerifyNet: Secure and verifiable federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 911–926, 2020.
- [26] S. Garg et al., "Experimenting with zero-knowledge proofs of training," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2023, pp. 1880–1894.
- [27] Y. Liu et al., "Provenance of training without training data: Towards privacy-preserving DNN model ownership verification," in *Proc. ACM Web Conf. 2023*, 2023, pp. 1980–1990.
- [28] T. Dong et al., "RAI2: Responsible identity audit governing the artificial intelligence," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2023, pp. 1–18.
- [29] J. Liu and L. Zhang, "MatProofs: Maintainable matrix commitment with efficient aggregation," in *Proc. ACM SIGSAC Conf. Computer Commun. Secur.*, 2022, pp. 2041–2054.
- [30] S. Srinivasan, A. Chepurnoy, C. Papamanthou, A. Tomescu, and Y. Zhang, "HyperProofs: Aggregating and maintaining proofs in vector commitments," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 3001–3018.
- [31] C. Papamanthou, E. Shi, and R. Tamassia, "Signatures of correct computation," in *Proc. 10th Theory Cryptography Conf.*, 2013, pp. 222–242.
- [32] J. Groth, M. Kohlweiss, M. Maller, S. Meiklejohn, and I. Miers, "Updatable and universal common reference strings with applications to zk-SNARKs," in *Proc. 38th Annu. Int. Cryptol. Conf.*, 2018, pp. 698–728.
- [33] H. Deng et al., "POLO: Proof-of-learning and proof-of-ownership at once with chained watermarking," *CoRR*, vol. abs/2505.12296, pp. 1–19, 2025.
- [34] G. K. Nayak, K. R. Mopuri, V. Shaj, V. B. Radhakrishnan, and A. Chakraborty, "Zero-shot knowledge distillation in deep networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 4743–4751.
- [35] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 14774–14784.
- [36] S. Wang, K. Gai, J. Yu, L. Zhu, W. Meng, and B. Xiao, "EASER: Embedding aggregation-based heterogeneous models training in vertical federated learning," *IEEE Trans. Mobile Comput.*, vol. 24, no. 12, pp. 12903–12917, Dec. 2025.
- [37] T. Peng et al., "Privacy-preserving truth discovery based on secure multi-party computation in vehicle-based mobile crowdsensing," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 7, pp. 7767–7779, Jul. 2024.
- [38] X. Cao, J. Jia, and N. Z. Gong, "IPGuard: Protecting intellectual property of deep neural networks via fingerprinting the classification boundary," in *Proc. ACM Asia Conf. Computer Commun. Secur.*, 2021, pp. 14–25.
- [39] T. Cong, X. He, and Y. Zhang, "SSLGuard: A watermarking scheme for self-supervised learning pre-trained encoders," in *Proc. ACM SIGSAC Conf. Computer Commun. Secur.*, 2022, pp. 579–593.
- [40] X. Wen, Y. Li, W. Jiang, and Q. Xu, "On function-coupled watermarks for deep neural networks," *IEEE J. Emerging Selected Topics Circuits Syst.*, vol. 14, no. 4, pp. 608–619, 2024.
- [41] A. Dziedzic, N. Dhawan, M. A. Kaleem, J. Guan, and N. Papernot, "On the difficulty of defending self-supervised learning against model extraction," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 5757–5776.
- [42] A. Dziedzic et al., "Dataset inference for self-supervised models," in *Proc. 36th Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 12058–12070.
- [43] H. Jia, H. Chen, J. Guan, A. S. Shamsabadi, and N. Papernot, "A zest of lime: Towards architecture-independent model distances," in *Proc. 10th Int. Conf. Learn. Representations*, 2022, pp. 1–17.
- [44] K. Gai et al., "MFL-Owner: Ownership protection for multi-modal federated learning via orthogonal transform watermark," in *Proc. Conf. Assoc. Advance. Artif. Intell.*, 2025, vol. 39, no. 3, pp. 3049–3058.
- [45] Y. Zhang, D. Zeng, J. Luo, Z. Xu, and I. King, "A survey of trustworthy federated learning with perspectives on security, robustness and privacy," in *Proc. Companion: Companion Proc. ACM Web Conf.*, 2023, pp. 1167–1176.
- [46] C. Niu, F. Wu, S. Tang, S. Ma, and G. Chen, "Toward verifiable and privacy preserving machine learning prediction," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 1703–1721, May-Jun. 2022.
- [47] X. Guo et al., "VeriFL: Communication-efficient and fast verifiable aggregation for federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 1736–1751, 2021.
- [48] X. Liu, B. Wu, X. Yuan, and X. Yi, "Leia: A lightweight cryptographic neural network inference system at the edge," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 237–252, 2021.
- [49] Y. Chang, H. Jiang, C. Lin, X. Huang, and J. Weng, "Towards understanding and enhancing security of proof-of-training for DNN model ownership verification," in *Proc. 34th USENIX Secur. Symp.*, 2025, pp. 7233–7250.
- [50] J. Liu, R. Zhang, S. Szyller, K. Ren, and N. Asokan, "False claims against model ownership resolution," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 6885–6902.



**Tianxiu Xie** is currently working toward the PhD degree in cyberspace security with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing, China. Her research interests include consortium blockchain, AI security, and encryption algorithm.



**Keke Gai** (Senior Member, IEEE) received the PhD degree from Pace University, USA. He is currently a professor with the School of Artificial Intelligence and with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing, China. He is also a professor with Zhongguancun Academy, Beijing. He has authored or coauthored five books, more than 200 peer-reviewed papers. His research interests include AI security, AI safety, cybersecurity, privacy computation, and blockchain.



**Liehuang Zhu** (Senior Member, IEEE) received the PhD degree in computer science from the Beijing Institute of Technology, Beijing, China, in 2004. He is currently a professor with the School of Computer Science & Technology, Beijing Institute of Technology, Beijing. He has authored or coauthored more than 400 peer-reviewed journal or conference papers, including more than ten IEEE/ACM Transactions papers, such as *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Industrial Informatics*, *IEEE Transactions on Vehicular Technology*, *IEEE Transactions on Smart Grid*, *Information Sciences*, *IEEE Network*, and *Computer & Security*. His research interests include security protocol analysis and design, wireless sensor networks, and cloud computing.



**Jing Yu** received the BS degree in automation science from Minzu University, China, in 2011, the MS degree in pattern recognition from Beihang University, China, in 2014, and the PhD degree from the University of Chinese Academy of Sciences, China, in 2019. She is currently a professor with the School of Information Engineering, Minzu University of China, Beijing, China. She has authored or coauthored more than 70 papers and has been granted four Best Paper awards. Her research interests mainly include cross-modal understanding and AI security.



**Qi Wu** (Senior Member, IEEE) is currently an associate professor with the University of Adelaide. He received the PhD degree in computer science from the University of Bath, U.K., in 2015. He was an ARC senior research associate with the Australian Centre for Robotic Vision. He has authored or coauthored more than 200 papers in decent venues. His research interests include object detection and vision-to-language problems, with a special focus on image captioning and visual question answering.